



CSE303: Statistics for Data Science [SPRING 2024]

Term Project Report

Submitted by:

Student ID	Student Name	Contribution Percentage
2022-1-60-013	Mohammed Shawqi Aftab	27.5%
2022-1-60-132	Jerin Anan Proma	27.5%
2022-1-60-153	Fatema Tuz Zannat	27.5%
2022-1-60-301	Kamrun Nahar	17.5%

ProjectLink :

https://colab.research.google.com/drive/1K_-2rNozC_brSYH6XVVyIWWFAXFnUrN?usp=sharing&authuser=1#scrollTo=gpbPvnJ-Bmv

1. Introduction

The project involved analyzing weather data from Dhaka and Delhi covering May 1, 2019, to April 30, 2024. At the beginning of this project, the main focus was on data preprocessing to make a smooth path for making prediction and classification models. Then it turned to comprehensive data visualization to unveil seasonal trends, anomalies, and correlations, thus setting the stage for subsequent analysis.

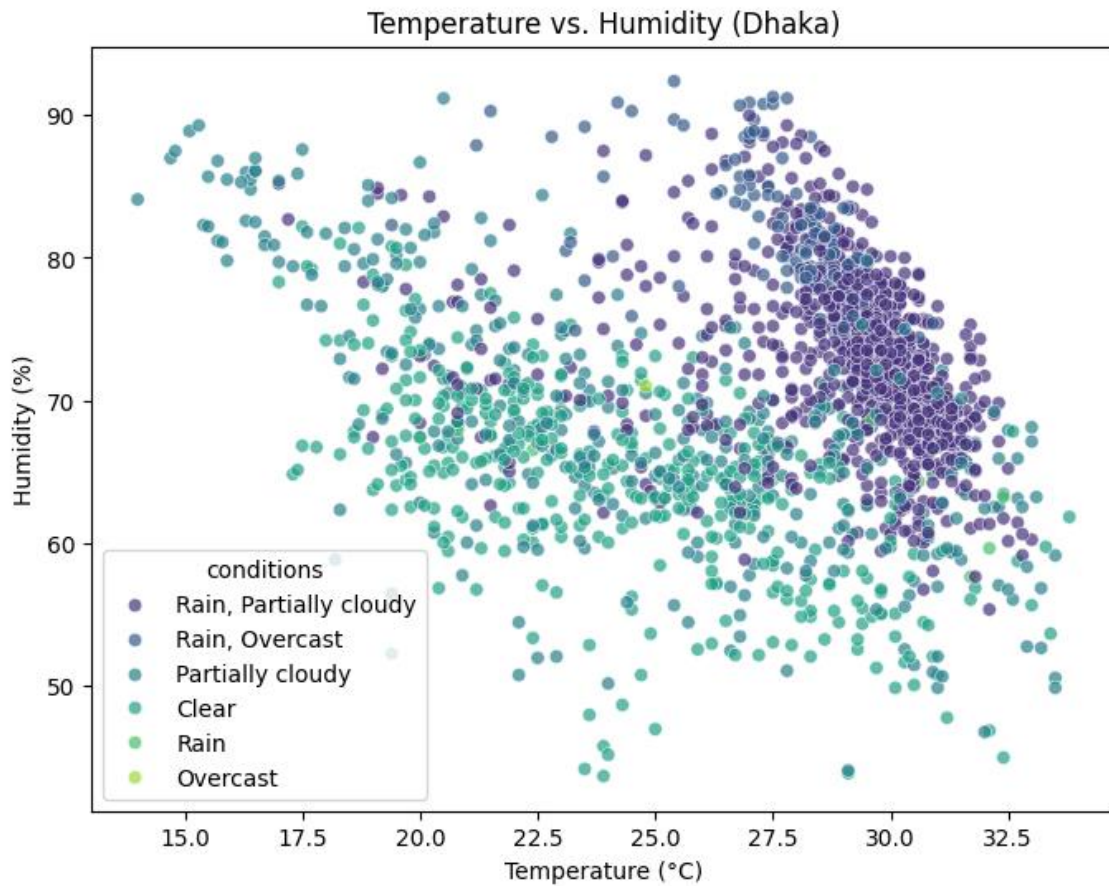
For temperature and rainfall prediction different Regression Models were employed, including Linear Regression and Random Forests. Different models showed different predictions though they followed a similar trend.

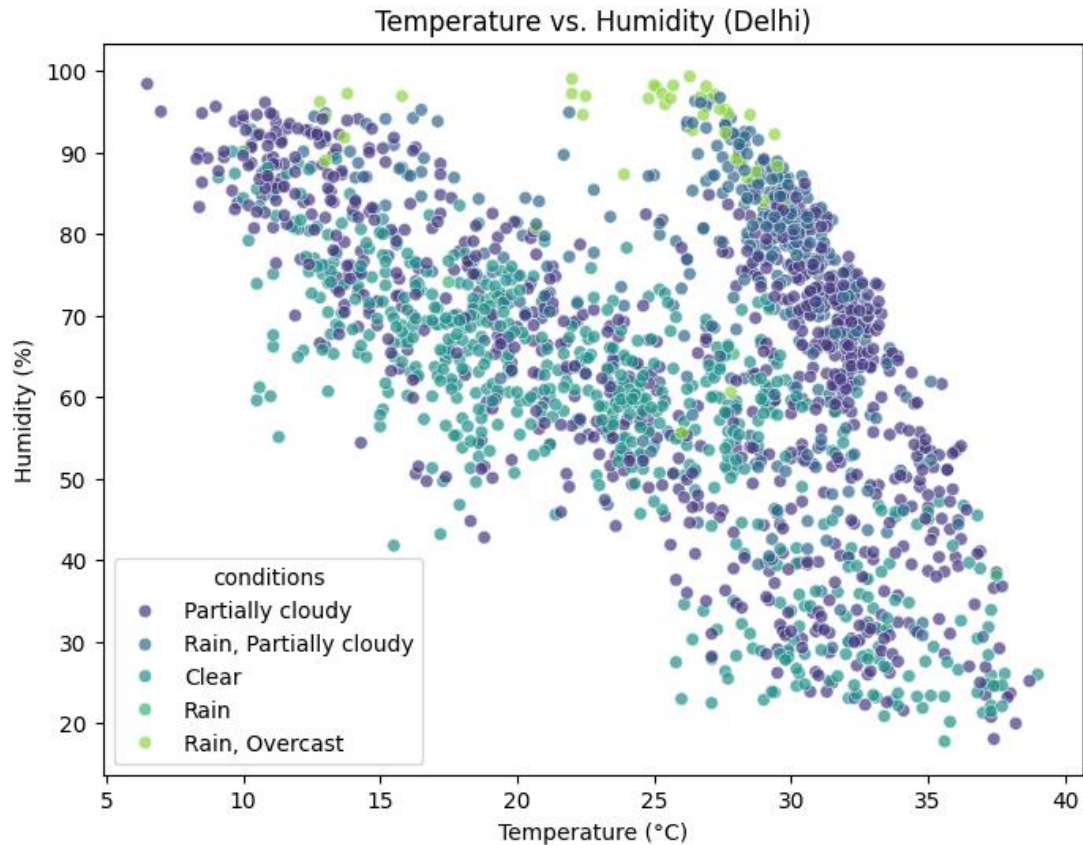
Concurrently, Classification Models such as Logistic Regression and Support Vector Machines were utilized to predict cities based on input features. These models achieved high accuracy, precision, and recall in city classification tasks, effectively predicting the class between Dhaka and Delhi.

In summary, the project showcased the efficacy of machine learning techniques in weather prediction and city classification tasks. Through various data visualization, preprocessing, and model development, valuable insights were collected into weather data analysis and machine learning methodologies. The project not only provided a deeper understanding of weather patterns in Dhaka and Delhi but also underscored the importance of strong analytical approaches in tackling complex datasets.

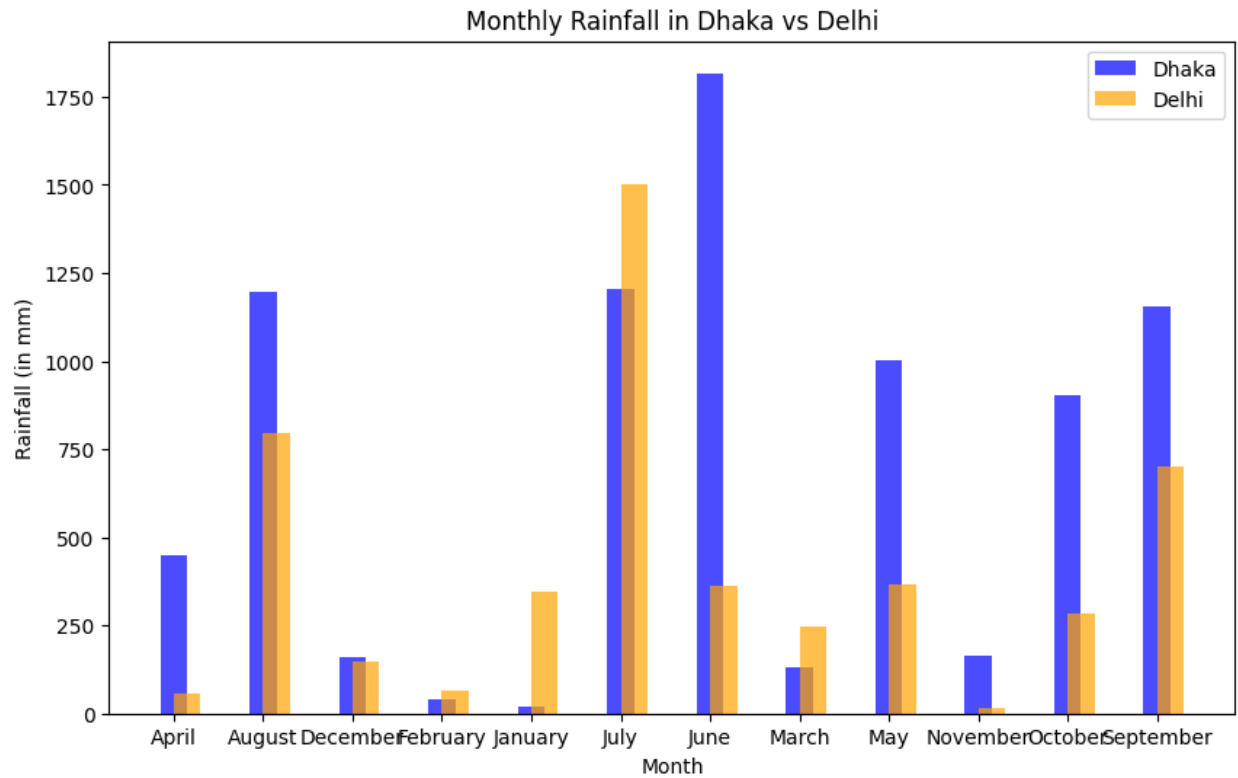
2. Dataset Characteristics and Exploratory Data Analysis

We were given two datasets, Delhi_Dataset and Dhaka_Dataset. Both datasets are weather-related, with various features that include meteorological measurements and conditions. Both contain 1827 rows and 33 columns. The datasets include 24 numerical attributes and 9 categorical attributes.

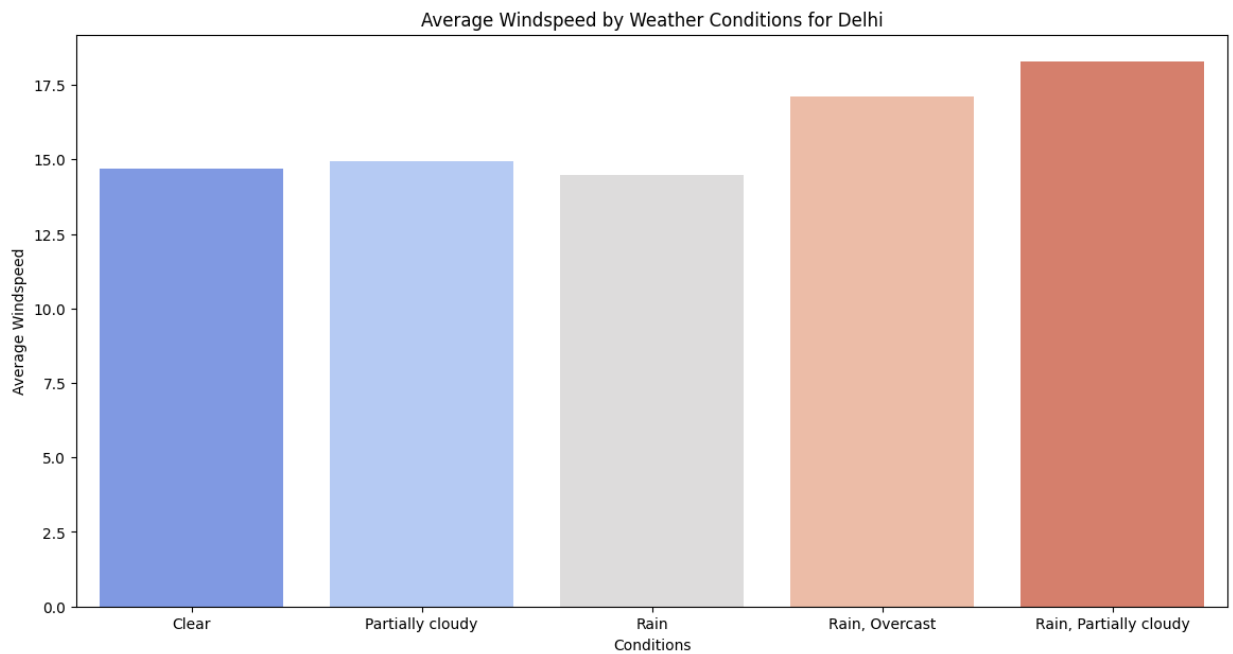
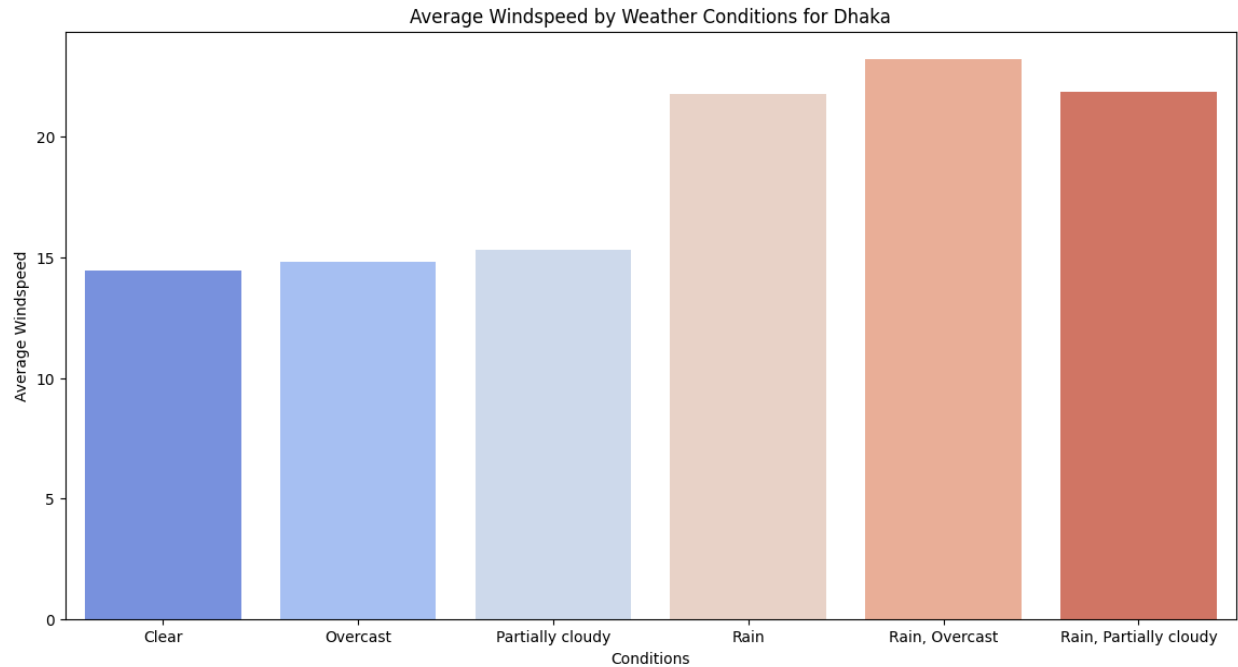




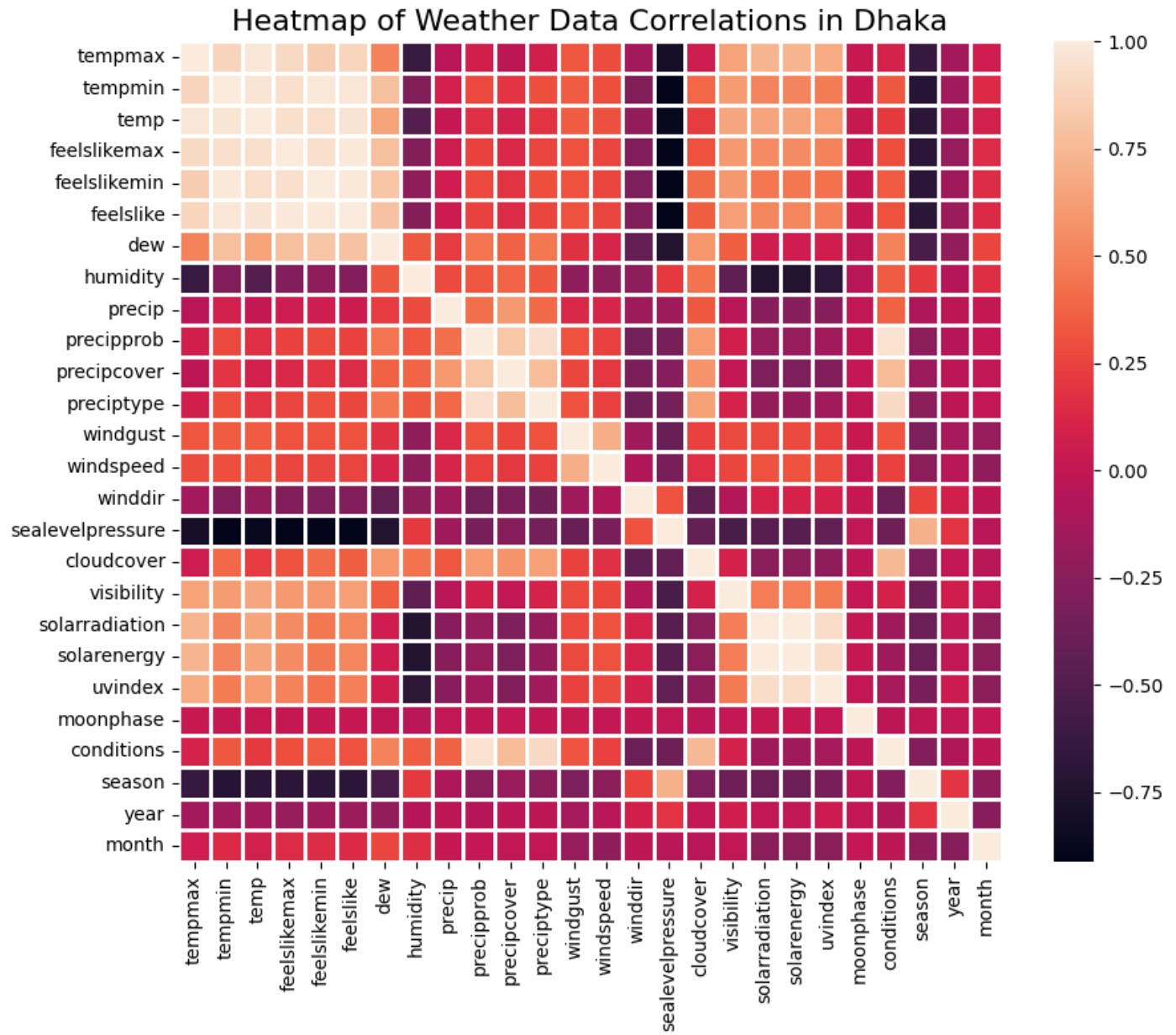
The two scatter plots show the relationship between temperature and humidity in two different cities: Dhaka and Delhi. Both graphs show an inverse relationship between temperature and humidity. As temperature increases, humidity generally decreases, and vice versa. So, we can say that temperature and humidity have an inverse relationship between them. The temperature in Dhaka ranges from around 15°C to 32.5°C. While the temperature in Delhi ranges from around 5°C to 40°C, indicating a broader range with potentially more extreme temperatures. Humidity in Dhaka ranges from around 50% to 90%. The data points are denser in the mid-range temperatures (20°C to 30°C) with higher humidity (60% to 80%). On the other hand, humidity in Delhi ranges from around 20% to 100%. The data points cover a wider range of humidity, and the distribution is more spread out. For both cities, rainy conditions are more frequent in areas with higher humidity. In conclusion we can say that Delhi experiences a wider range of both temperature and humidity compared to Dhaka.

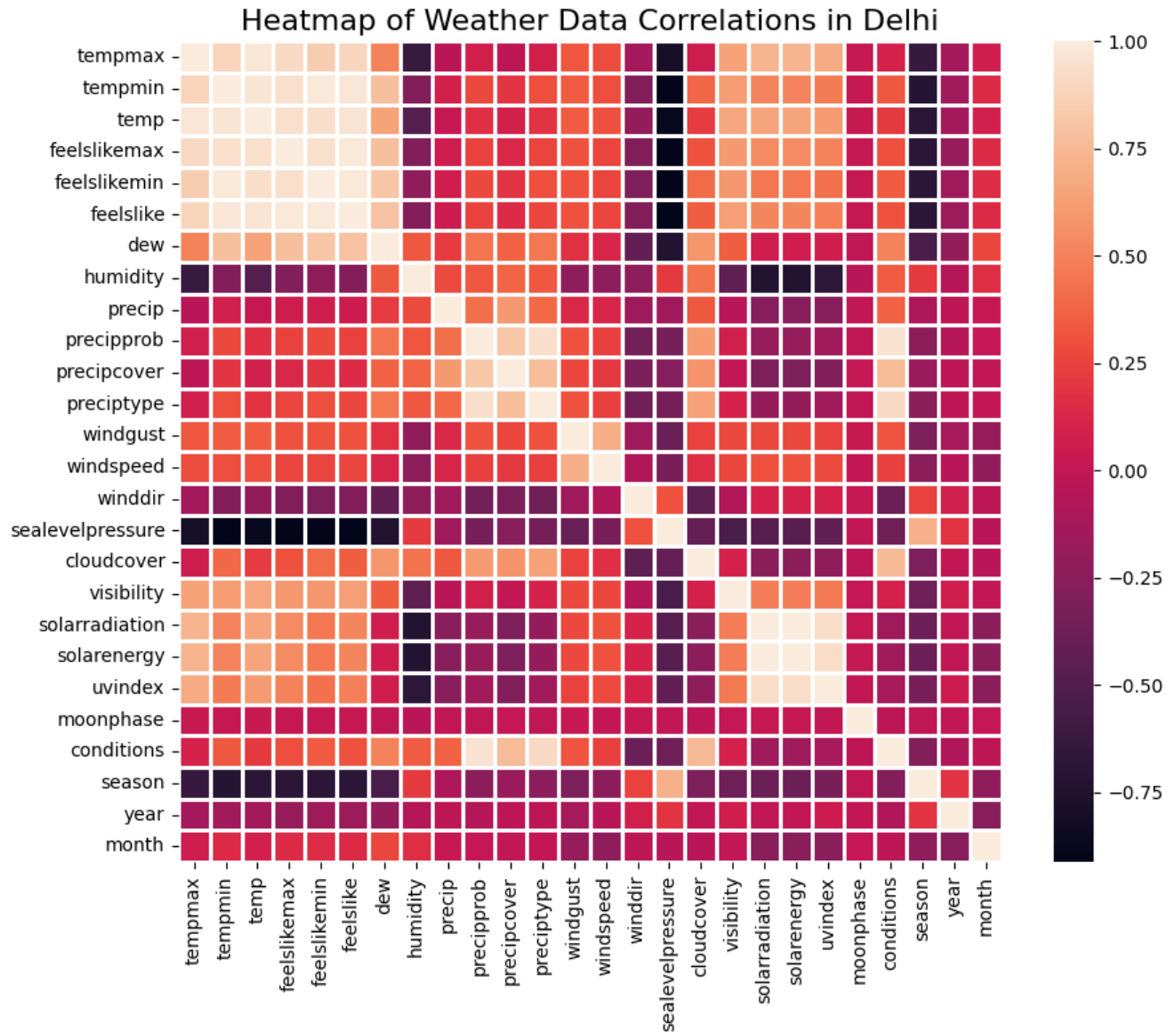


The bar chart represents monthly rainfall in the respective cities. For Dhaka highest rainfall occurs in June (~1750 mm) followed by August and July. And for Delhi it is July (~1450 mm) followed by August and January. By analyzing further, we can see that Delhi's rainfall values are generally lower than Dhaka's for comparable months. These differences could be because of the geographic and climatic difference between the two cities.



The two bar graphs depict the average wind speed under different weather conditions for Dhaka and Delhi. Dhaka generally experiences higher average wind speeds under all considered weather conditions compared to Delhi. The highest wind speeds in Dhaka occur during combined rainy and overcast or partially cloudy conditions, reaching up to around 20-22 units. while Delhi's wind speeds peak under rainy and partially cloudy conditions, peaking around 18 units.

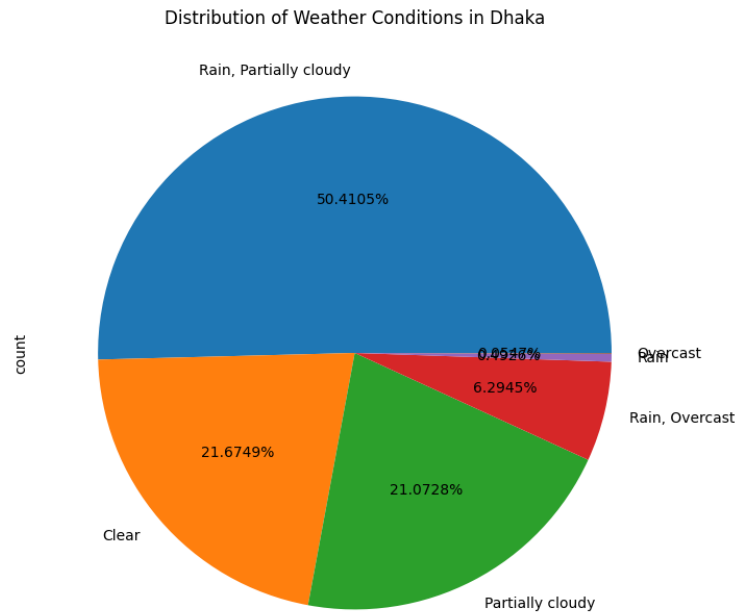


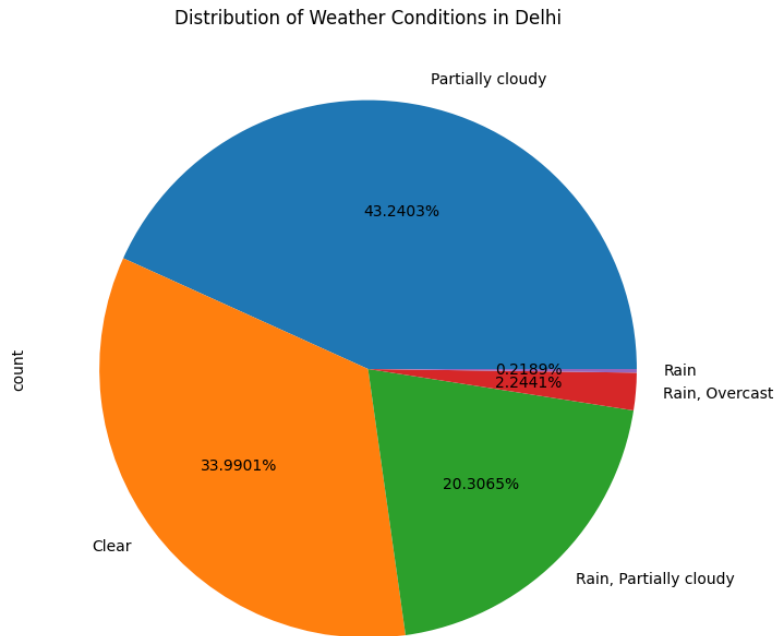


These heatmaps are showing the correlations between various weather data parameters in Dhaka and Delhi. Correlation values range from -1 (strong negative correlation) to 1 (strong positive correlation), with 0 indicating no correlation. The color scale from dark purple (negative correlation) to dark orange (positive correlation) to visualize these relationships. Both Dhaka and Delhi show high positive correlations among temperature-related variables (tempmax, tempmin, temp, feelslikemax, feelslikemin, feelslike, dew) which is expected, as these variables are closely related to each other. There is a negative correlation between humidity and temperature-related variables in both cities and wind-related variables (windgust, windspeed, winddir) show mixed correlations with other parameters. Solar radiation and solar energy are positively correlated with temperature and

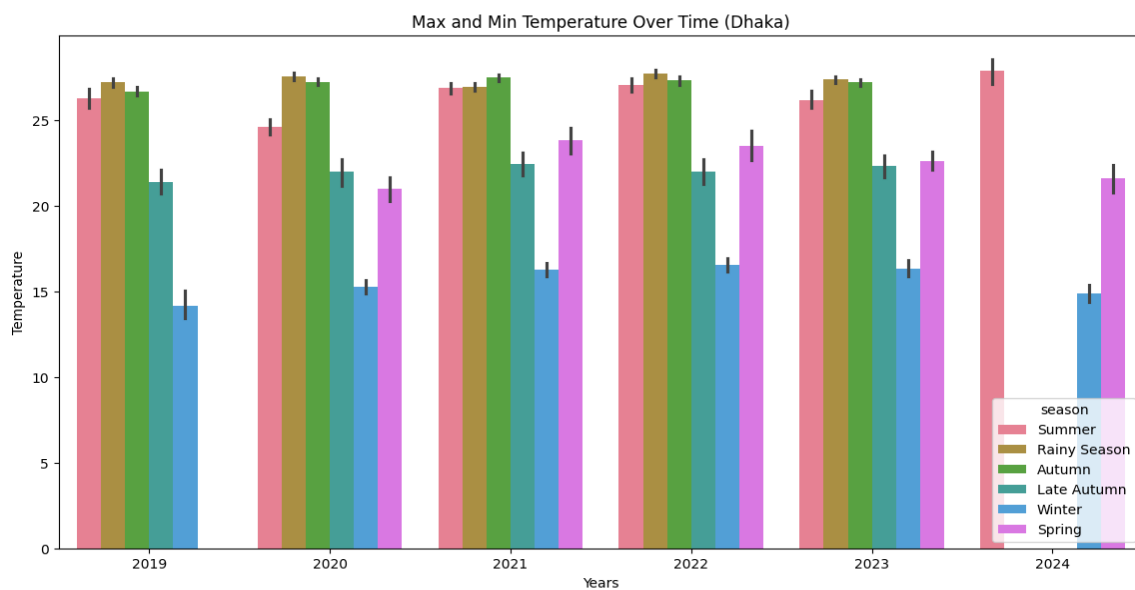
negatively correlated with cloud cover in both cities, as clearer skies allow more solar radiation.

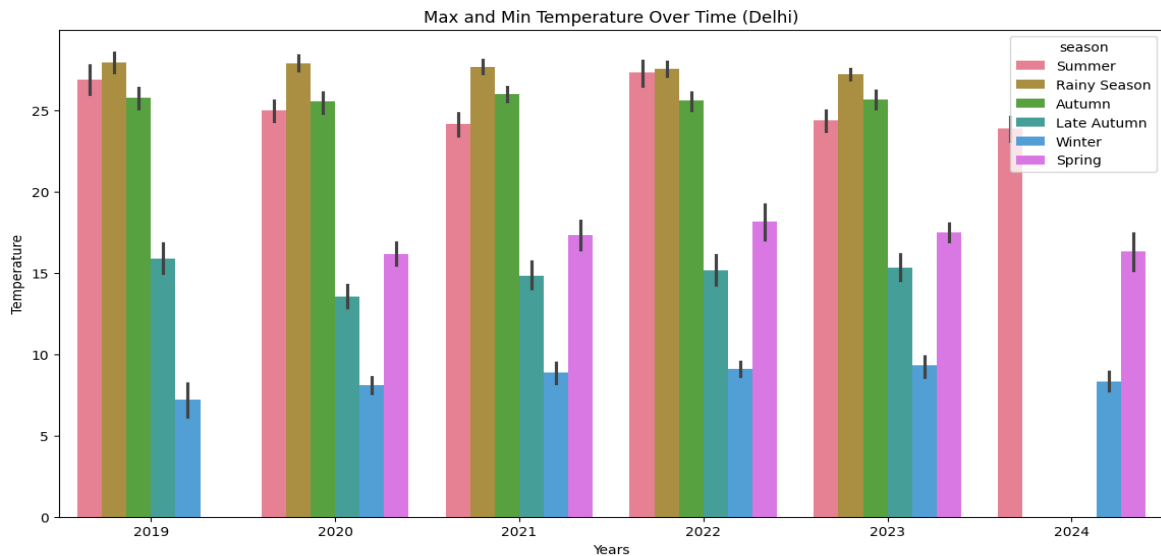
While both cities exhibit similar correlation patterns among weather parameters, the correlation strengths vary between Dhaka and Delhi. Such as the correlation between temperature variables and solar energy/radiation is stronger in Dhaka compared to Delhi. Visibility shows a higher positive correlation with temperature and a negative correlation with humidity in Dhaka compared to Delhi.



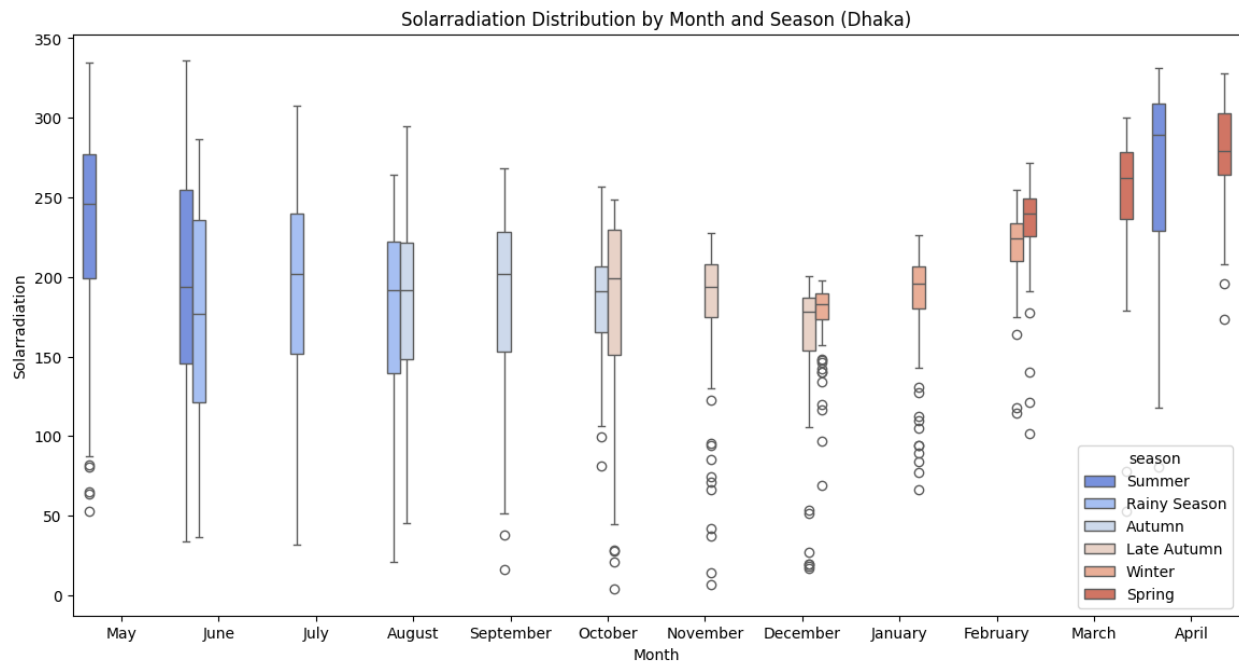


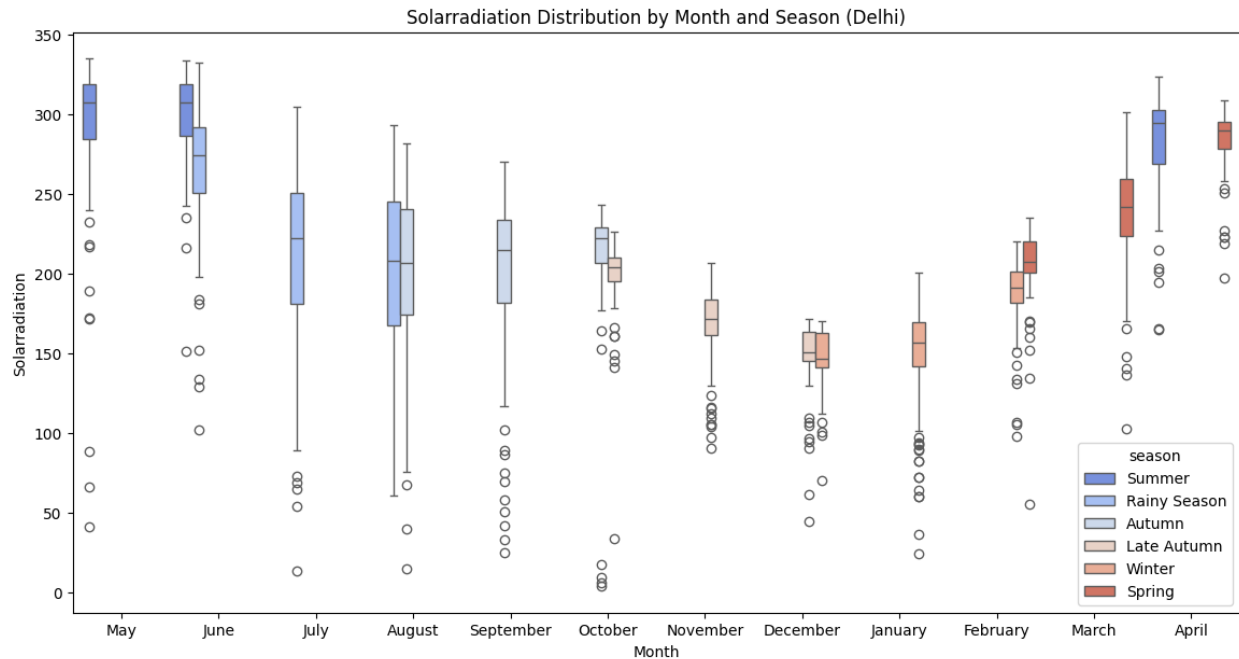
By analyzing the pie charts we can understand the distribution of different weather conditions in Dhaka and Delhi. Each segment represents the percentage of time each weather condition is observed. In Dhaka Rain, Partially cloudy is observed most (50.41%) while in Delhi Partially cloudy is observed most (43.24%). Dhaka experiences clear weather 21.67% of the time. And Delhi has clear weather more often, at 33.99%. In short, Dhaka tends to have more rain-associated weather, often combined with cloudiness, whereas Delhi experiences clearer and less rainy weather more frequently.



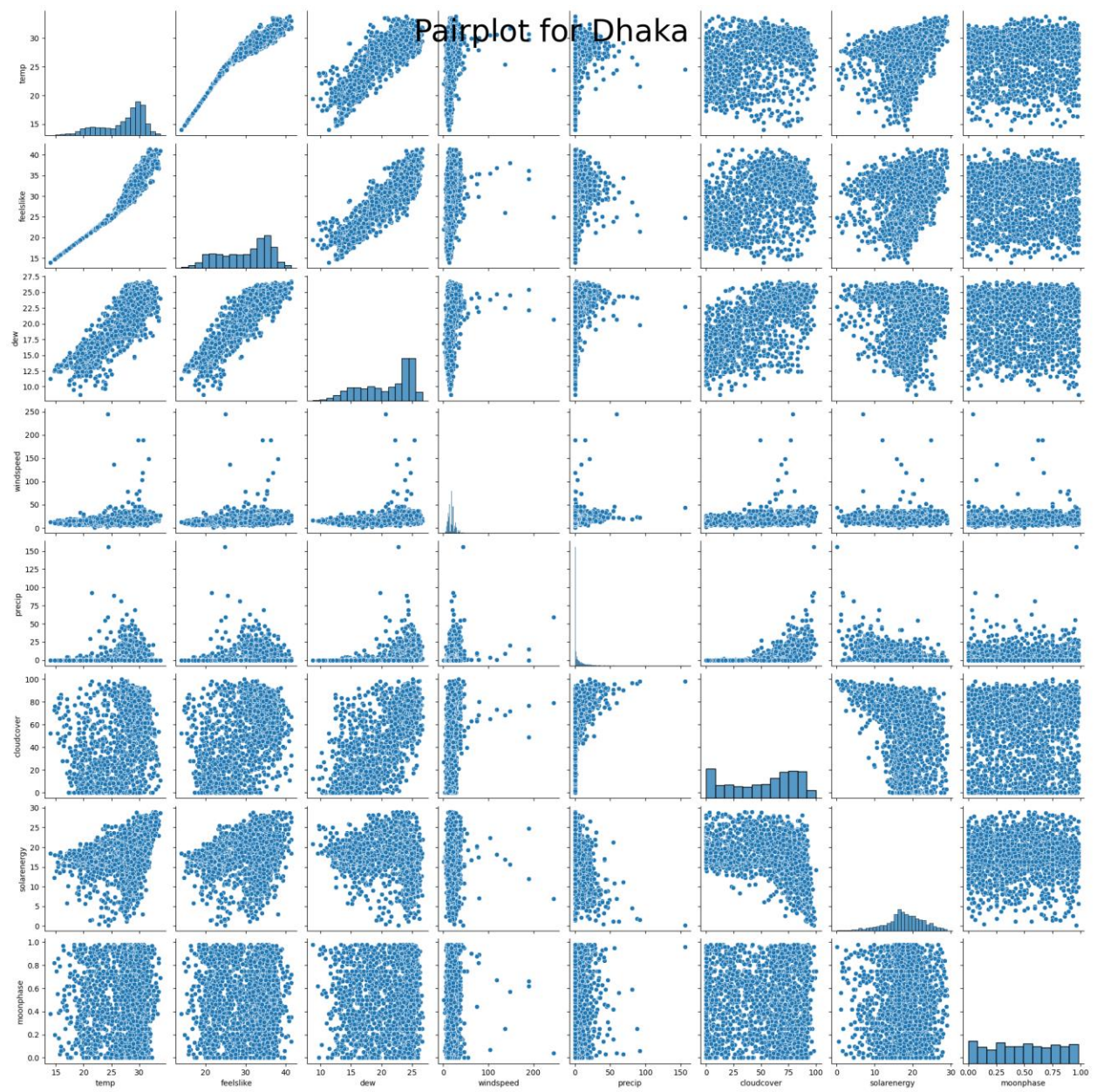


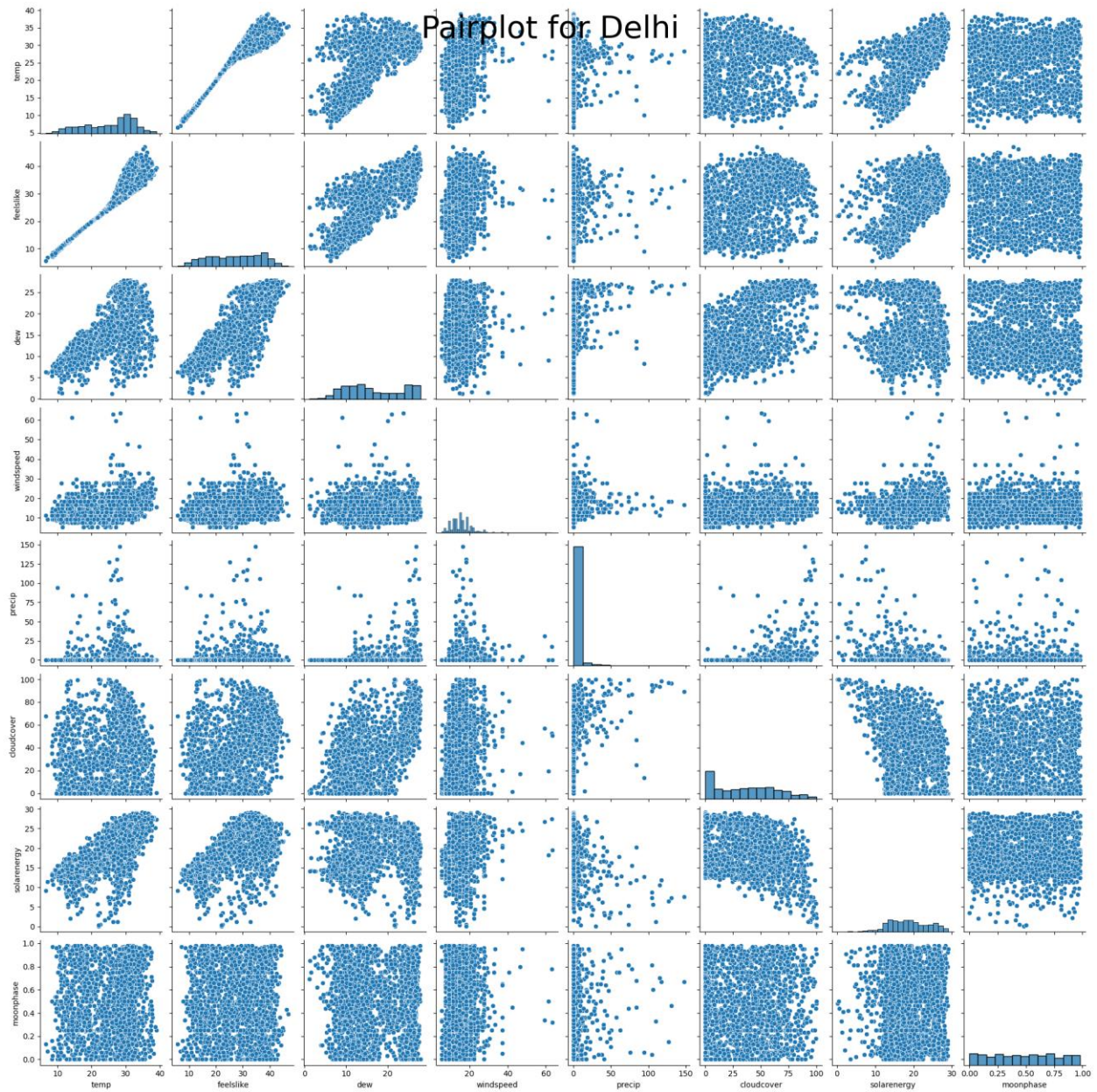
The two graphs show the maximum and minimum temperatures over time for different seasons in Dhaka and Delhi from 2019 to 2024. Both Dhaka and Delhi show distinct seasonal temperature patterns with high temperatures in summer and low in winter. Delhi experiences more extreme winter temperatures and a greater variation over the years compared to Dhaka. An upward trend in temperatures is observed in both cities, reflecting possible climate change impacts.





By observing the boxplots we can see that Both cities show significant seasonal variation in solar radiation. In both cities, summer months (May, June) tend to have the highest solar radiation, while winter months (November, December, January) have the lowest. In both cities, summer months (May, June) tend to have the highest solar radiation, while winter months (November, December, January) have the lowest. Delhi tends to have slightly higher solar radiation values compared to Dhaka, especially noticeable during summer months. These observations highlight the seasonal and monthly patterns of solar radiation in Dhaka and Delhi, with notable differences in the magnitude and variability between the two cities.





By analyzing the pair plots we can see that Both cities show a strong linear relationship between temperature and the "feels like" temperature, indicating that as the actual temperature rises, the perceived temperature increases as well. There is a positive correlation between temperature and dew point, suggesting that higher temperatures are associated with higher dew points for both cities. For Dhaka, wind speed has a wider range and some higher outliers compared to Delhi. Overall, the pair plots reveal similarities in the weather patterns of Dhaka and Delhi as well as unique aspects like the distribution of wind speeds and solar radiation exposure.

3. Machine Learning Models

=> For **predicting** the temperature and rainfall in Dhaka and Delhi we have used several models. The models are-

1. **Linear Regression Mode:** It is a supervised model which predicts numeric values.
2. **Ridge** : It is less restrictive on coefficients than Lasso, which tends to lead to lower bias but possibly higher variance.
3. **Lasso** : It regularizes data very strongly to low bias and low variance.
4. **Elastic Net** : It is the combination of L1 and L2 regression.
5. **Random Forest** : Random forest is a commonly-used machine learning model that combines the output of multiple decision trees to reach a single result.

=> For the **Classification models** we have used both a Logistic Regression Model and a Support Vector Machine.

1. **SVM:** This is a supervised learning model that can find different decision boundaries and classify data.
- 2.
3. **Logistic Regression:** This is a supervised learning model that can classify data into binary classes.

4. Data Preprocessing

=> At first, we combined the two-dataset using **concat** since we will need both dataset for classification models. Since both datasets have the same columns, we used concat to combine.

=> Then we dropped some insignificant columns like (stations, icons, sunset) etc. because they do not have any meaningful information related to the models we are working with.

=> Then we looked for columns with null values. We found two columns with null values (preciptype, severerisk). We dropped severerisk column since it seemed irrelevant for our desired model. Since precipetype column contained categorical data we used **fillna ()** to fill in the null values.

=> The datetime column was a string column with unique values in each row. So we calculated and added 3 new columns (season, month, year) from date which was helpful in building our models and data visualization purposes.

=> We used **LabelEncoder()** to transform categorical attributes like (preciptype, conditions) etc to numerical attributes.

=> We also did data normalization for better performance of the models. We used **StandardScaler()** for columns with no outliers. For columns with outliers we used **RobustScaler()** as it is less sensitive to them compared to other scalers.

=> For selecting the features from the large dataset's columns we used **RandomForestClassifier**.

5. Different Models

PREDICTION MODELS:

1. Linear Regression:

Linear Regression is a supervised learning model which interprets the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data. It is simple and interpretable.

=> **Parameters:** No parameters were used in this model

2. Ridge Regression:

Ridge Regression is a type of linear regression that includes L2 regularization. It helps to prevent overfitting by Adding a penalty to the size of the coefficients to prevent any one feature from dominating.

3. Lasso Regression:

Lasso Regression incorporates L1 regularization, which not only helps prevent overfitting but also performs feature selection by shrinking some coefficients to zero. This makes it useful when dealing with high-dimensional data.

=> **Parameters:** For both the Ridge and Lasso Regularization The alpha parameter was used.

Lower alpha Values: Implies less regularization, allowing the model to fit more closely to the training data. This can capture more variance in the data but may lead to overfitting.

Higher alpha Values: Implies more regularization, shrinking more coefficients to zero, which can lead to a simpler model that may underfit the data if alpha is too large.

4. Elastic Net Regression:

Elastic Net is a hybrid of Lasso and Ridge Regression. It uses both L1 and L2 regularization, combining their strengths. It is particularly useful when dealing with highly correlated predictors.

alpha: This is the regularization strength parameter.

- [1000.0, 10000.0, 100000.0]: These values represent different levels of regularization strength. A higher alpha value increases the amount of regularization, which can help prevent overfitting by shrinking the coefficients of less important features closer to zero. However, too high of an alpha value can lead to underfitting,

l1_ratio: This parameter controls the balance between L1 and L2 regularization.

- [0.0001, 0.0005, 0.0009]: These values specify the proportion of L1 regularization in the mix. An l1_ratio of 1.0 corresponds to pure Lasso regression (L1 regularization), while a ratio of 0 corresponds to pure Ridge regression (L2 regularization). Values between 0 and 1 provide a balance, with the given values leaning heavily towards L2 regularization.

5. Random Forest Regression:

Random Forest is an ensemble learning method that constructs multiple decision trees and merges their results. It captures complex non-linear relationships and is good at generalizing data, but it is less interpretable than linear models.

Parameters for Random Forest

I. n_estimators: This parameter specifies the number of trees in the random forest.

- [5, 10]: This means we are considering using either 5 or 10 trees in the forest. More trees generally improve performance but also increase computational cost.

II. max_features: This parameter defines the number of features to consider when looking for the best split.

- **'auto':** This will use the square root of the total number of features ($\sqrt{n_features}$).
- **'sqrt':** This is equivalent to 'auto', explicitly specifying the square root of the total number of features.
- Using fewer features can reduce overfitting and improve generalization.

III. max_depth: This parameter limits the maximum depth of the trees.

- [1, 2]: This means we are limiting the depth of each tree to a maximum of either 1 or 2 levels. Shallower trees are less complex and more generalized but might underfit the data.

IV. min_samples_split: This parameter specifies the minimum number of samples required to split an internal node.

- [5, 10]: This means a node must have at least 5 or 10 samples to be considered for splitting. Higher values prevent the model from learning overly specific patterns and help avoid overfitting.

V. min_samples_leaf: This parameter specifies the minimum number of samples required to be at a leaf node.

- [2, 4]: This means a leaf node must contain at least 2 or 4 samples. Higher values lead to more generalized models, preventing the model from learning noise in the data.

CLASSIFICATION MODELS:

In both models, we used:

1. RFE (Recursive Feature Elimination), which is a feature selection method that fits a model and removes the weaker features until the specified number of features is reached. Features are ranked by the model's `coef_` or `feature_importances_` attributes, and by recursively eliminating a small number of features per loop, RFE attempts to eliminate dependencies and collinearity that may exist in the model. This is part of the sklearn library.

The parameters that we used in RFE are:

Estimator: We pass a model through this parameter; this learns from the data and trains the model. The model

N_features_to_select: The number of features we want to select.

2. GridSearchCV which is a cross validation technique that we used to test our models with various values of their respective parameters.

3. Max_iter the amount of time the model will run the default is 100. This is the maximum iterations for a dataset to converge.

The parameters we used in GridSearchCV are:

Estimator: We pass a model through this parameter and get information

Param_grid : This is a python dictionary. We input the parameter names and different values of the parameters of the respective model to test all possible combinations of them and see which one is better.

Cv : Cv stands for cross validation. This parameter indicates how manyfold a cross validation will be. The default is 5.

Scoring: This determines the strategy of the cross validation. The default value of scoring is set for each model type. Some common attributes of scoring for classification models are accuracy, precision, recall etc. For Linear model we use `r2` or `neg_mean_squared_error`

SVM:

In GridSearchCV we used the parameters C and kernel for this model.

C: This is the inverse of regularization. C is proportional to (1/ Regularization) . The Default value of C is 1.0. If we increase the value C regularization will decrease then the model becomes overfit. If we decrease the value of C, then the model becomes underfit.

Kernel: In SVM a kernel is the function which determines the decision boundary. In our model we used the following:

Linear kernel: This is useful when the number of features is high relative to the number of samples, such as in text classification. It is defined as the standard dot product of the input vectors:

$$\mathbf{x}^T \cdot \mathbf{x}_j$$

Logistic Regression:

In GridSearchCV we used the parameters C and solver for this model.

C: This is the inverse of regularization. C is proportional to (1/ Regularization). The Default value of C is 1.0. If we increase the value C regularization will decrease then the model becomes overfit. If we decrease the value of C, then the model becomes underfit.

Solver: In logistic regression, the solver parameter specifies the optimization algorithm used to fit the model to the data. Different solvers use different techniques to minimize the loss function.

In our model we used

Liblinear: This solver updates one feature at a time while keeping the others fixed.

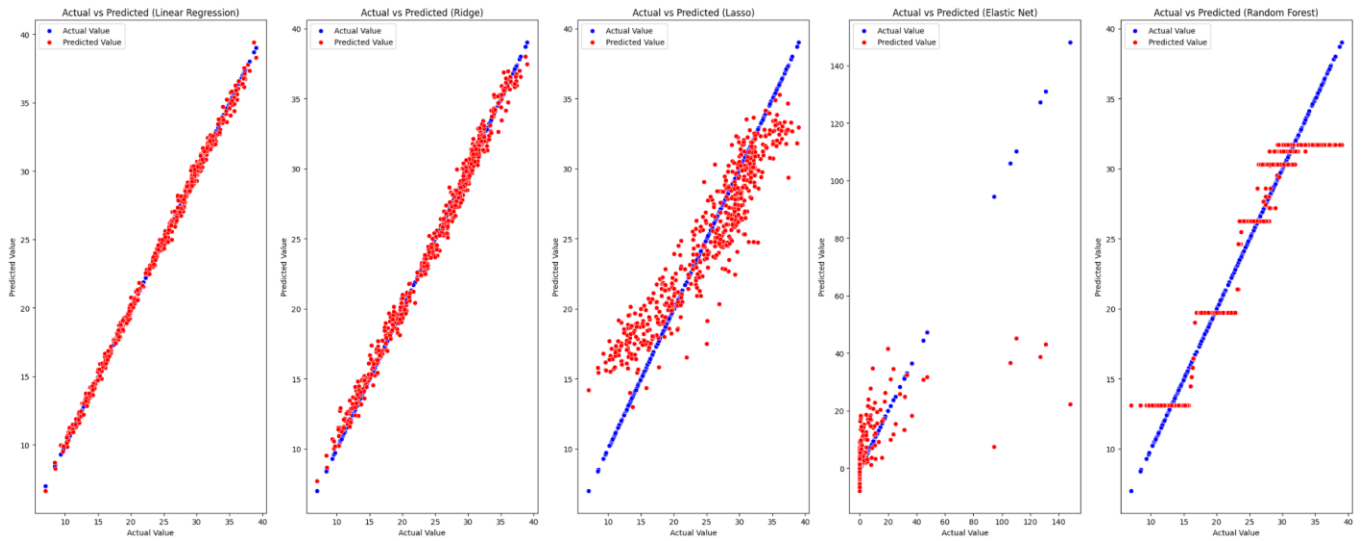
Iteratively minimizes the loss function by optimizing along one coordinate direction at a time.

Liblinear Supports both L1 (lasso) and L2 (ridge) regularization.

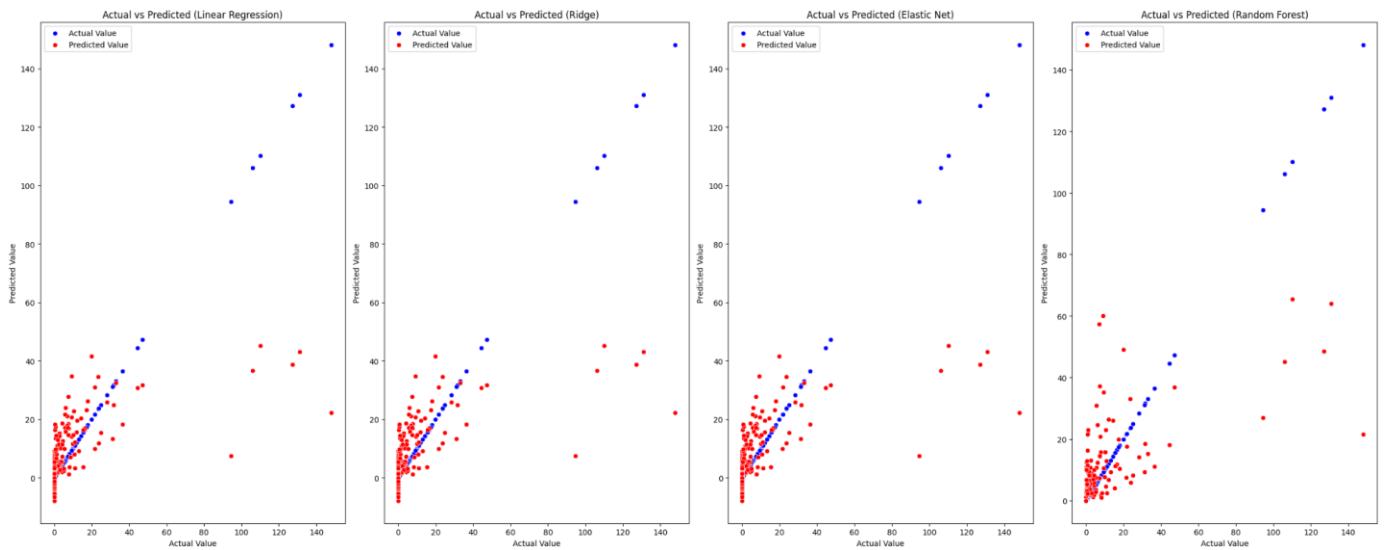
Lbfgs: This does not support L1 regularization but this is used widely in large datasets.

6. Performance Evaluation

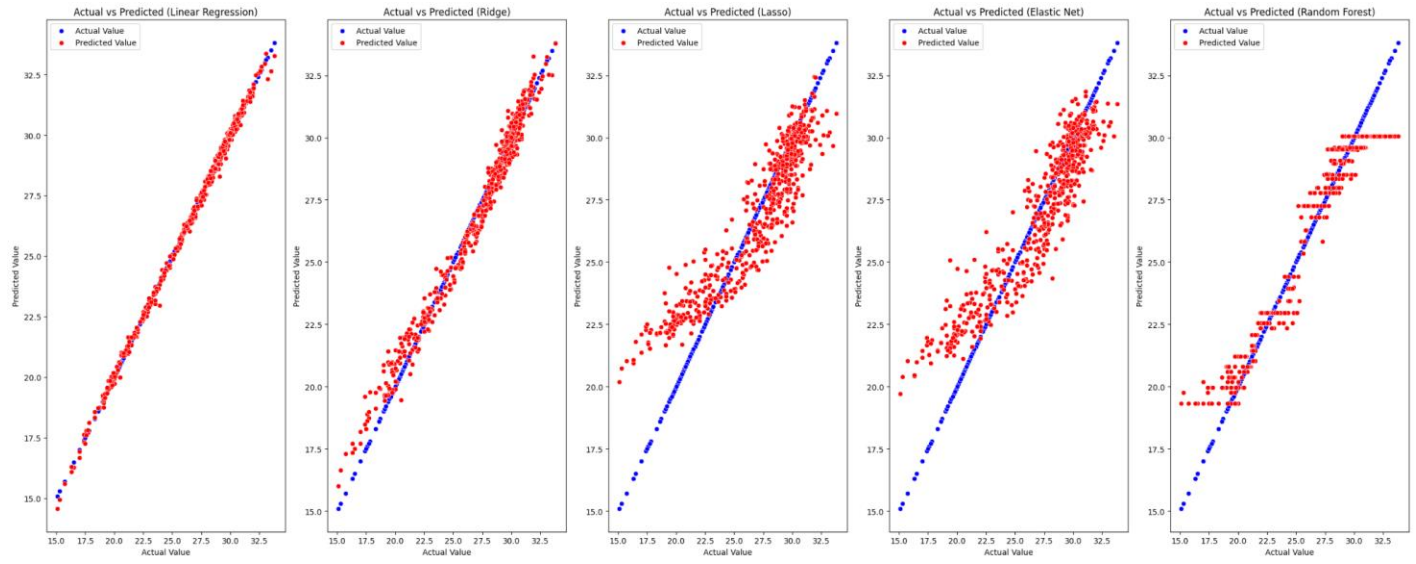
1. Temperature For Delhi



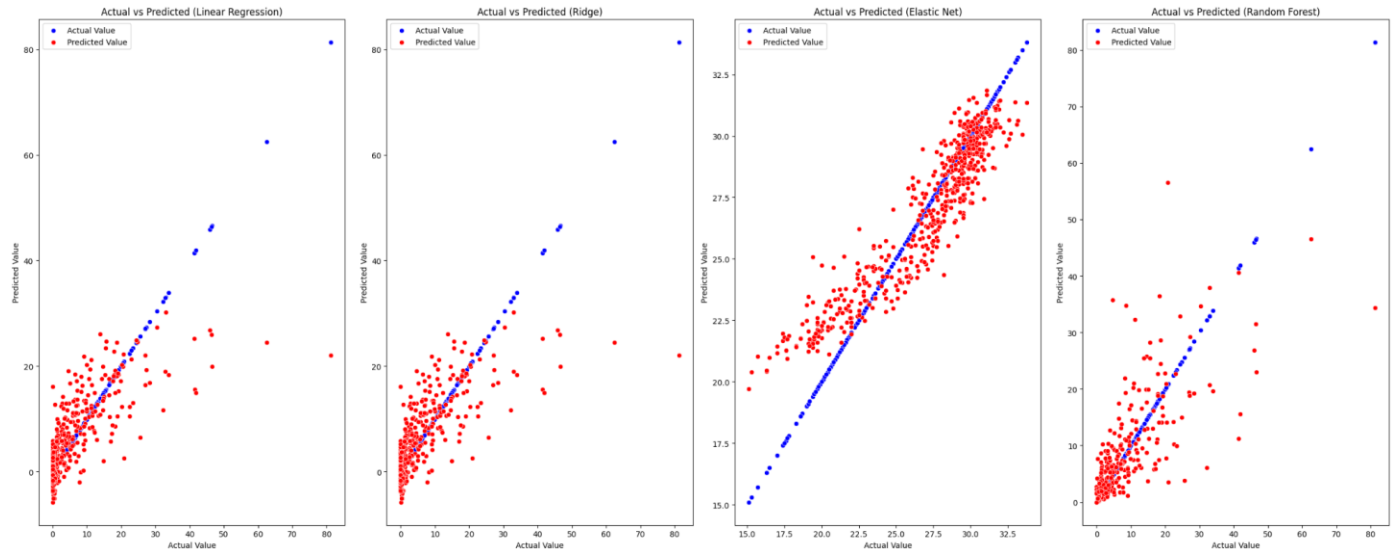
2. Rainfall For Delhi



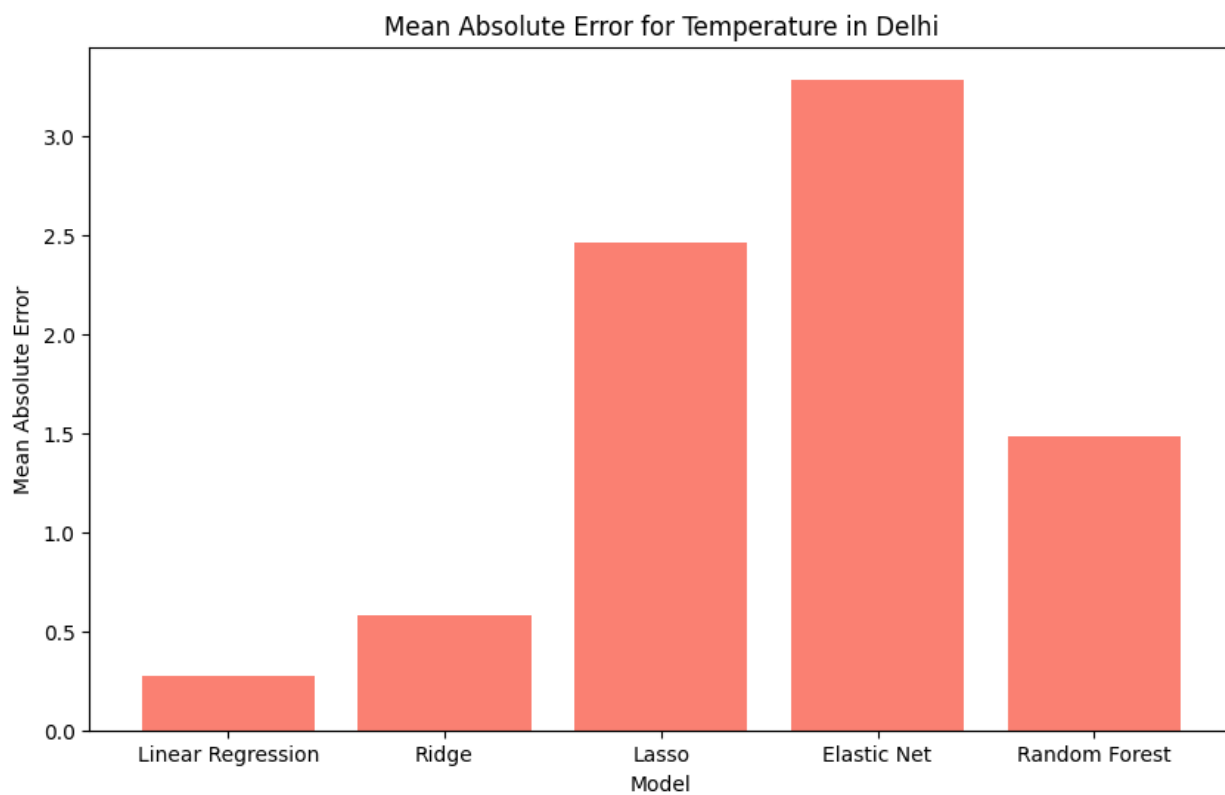
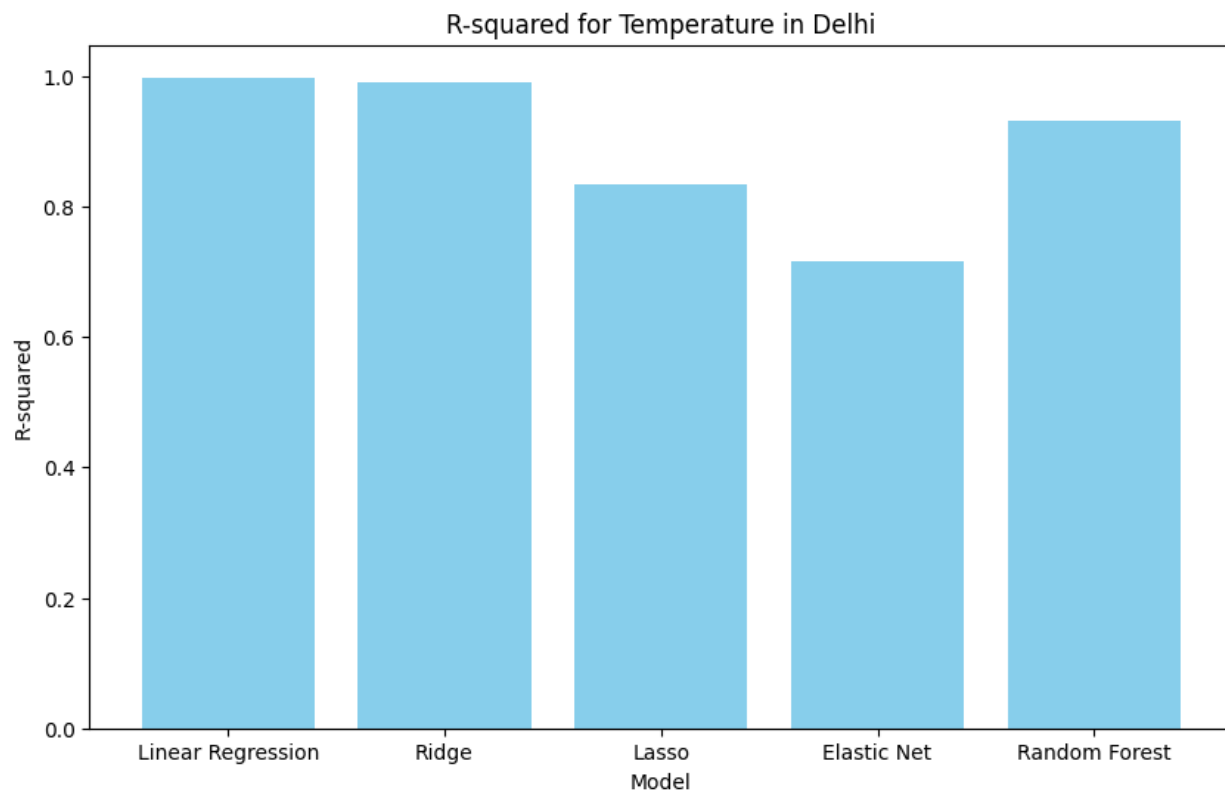
3. Temperature for Dhaka

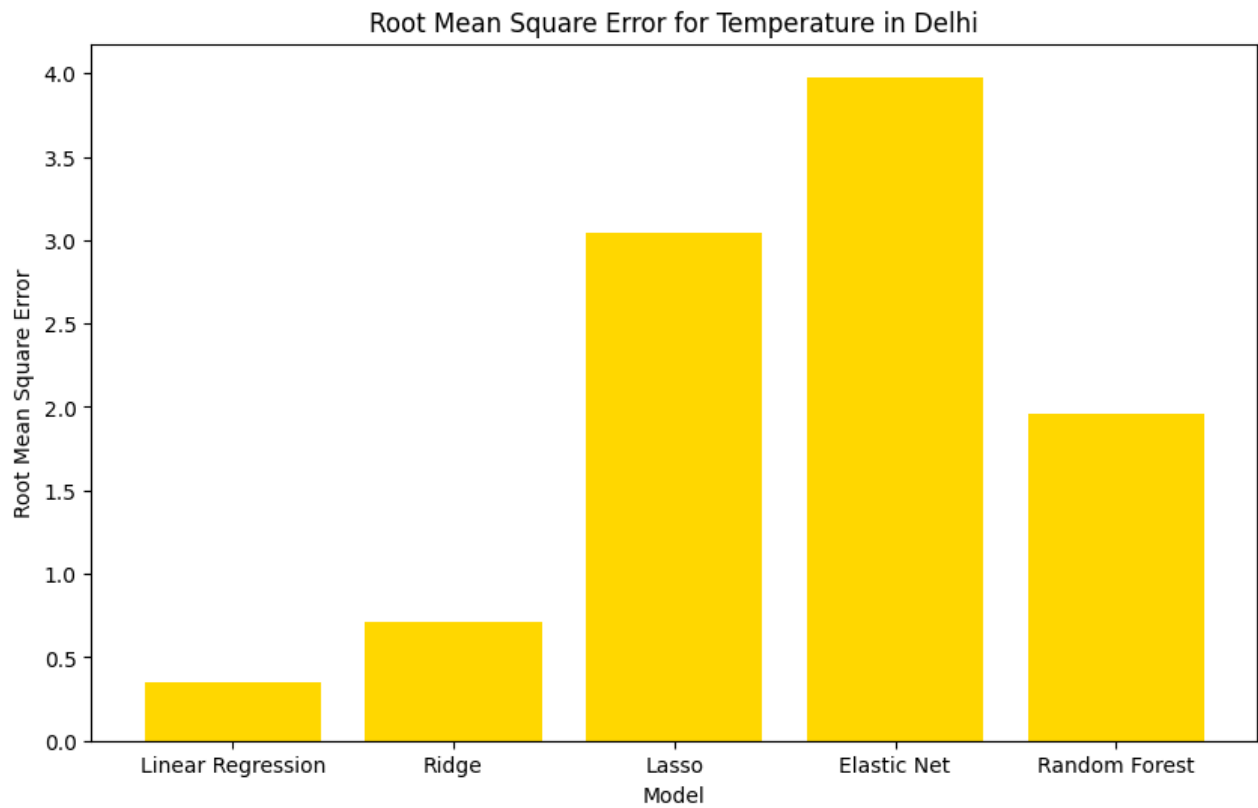
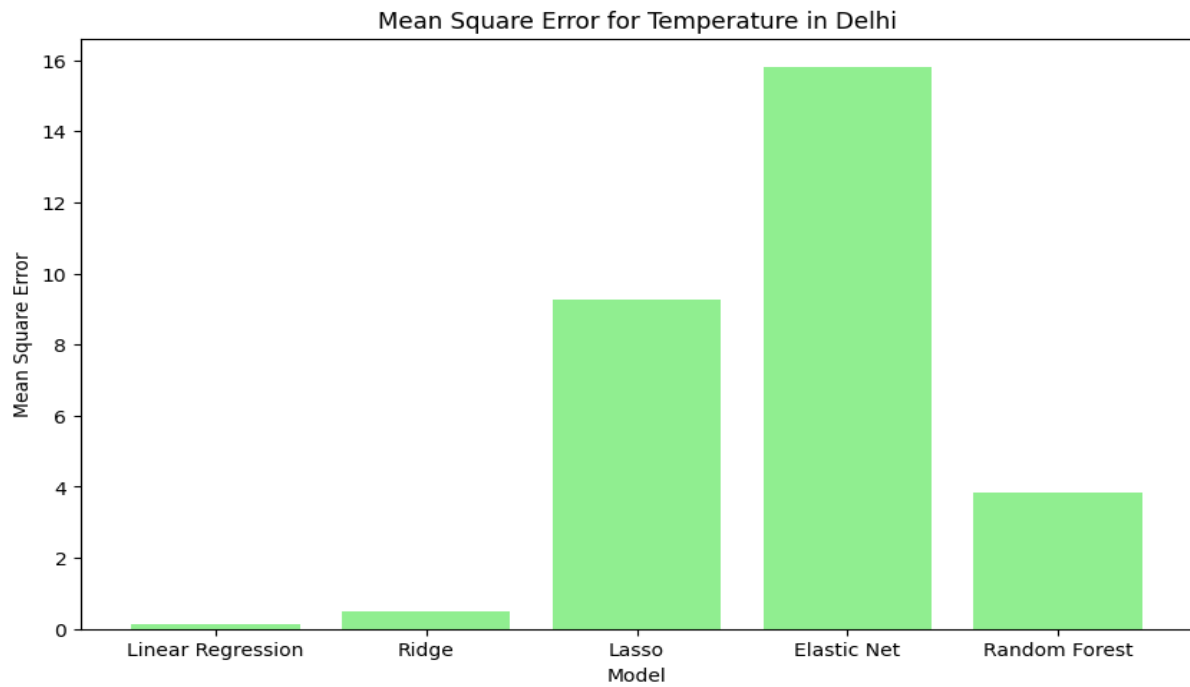


4. Rainfall for Dhaka

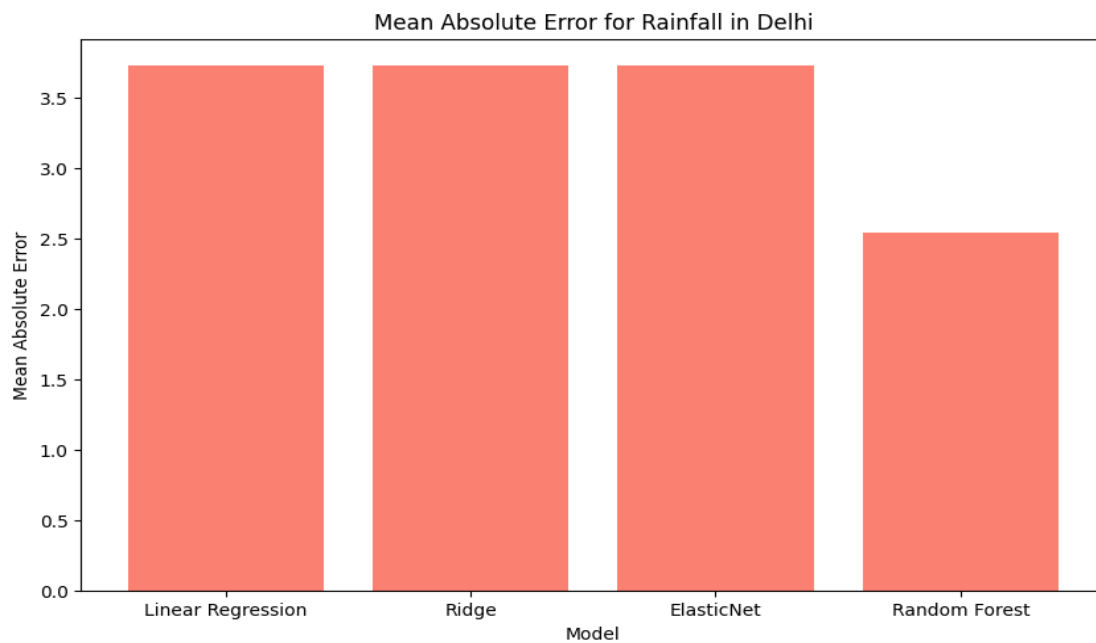
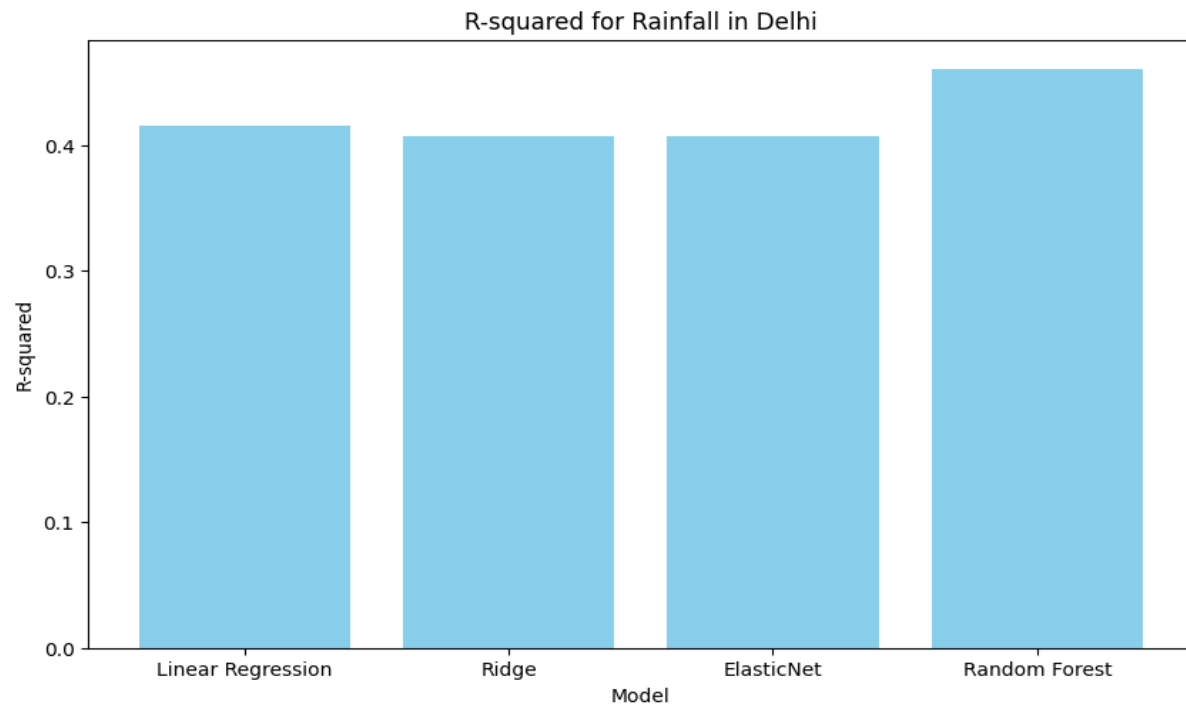


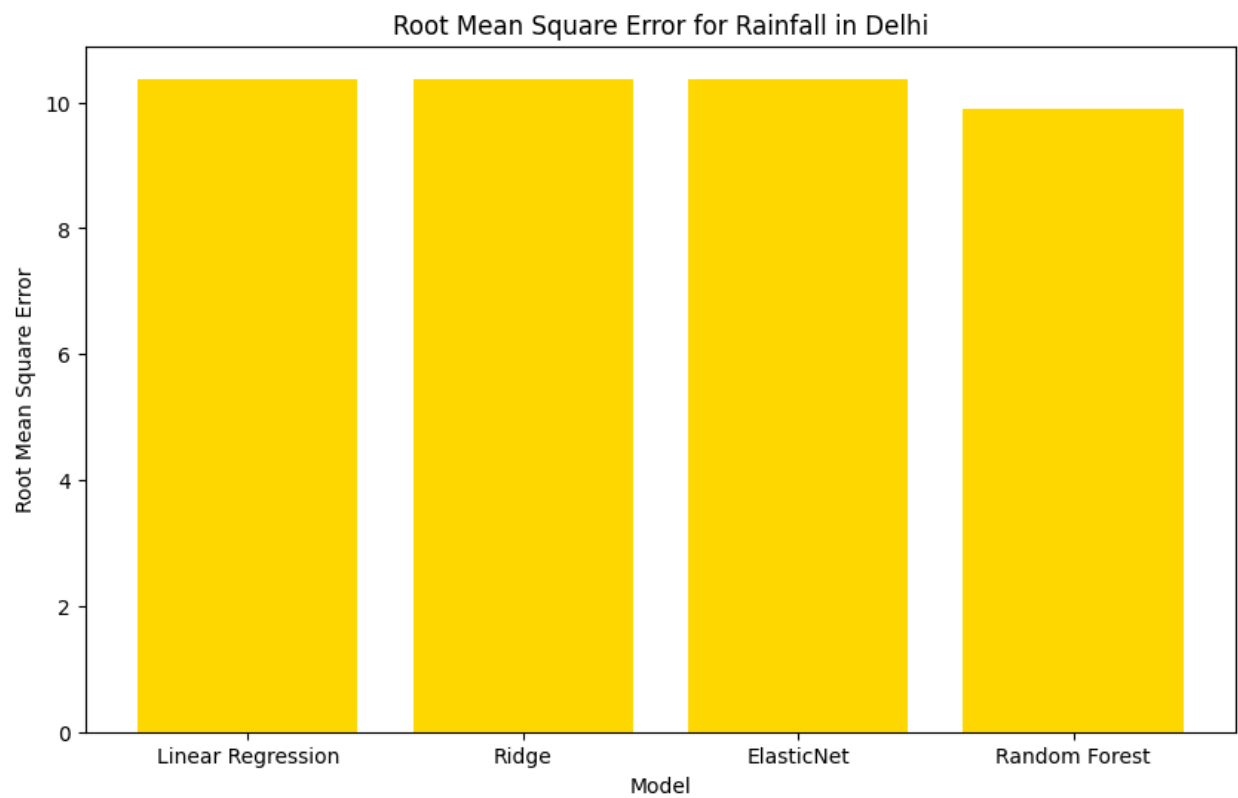
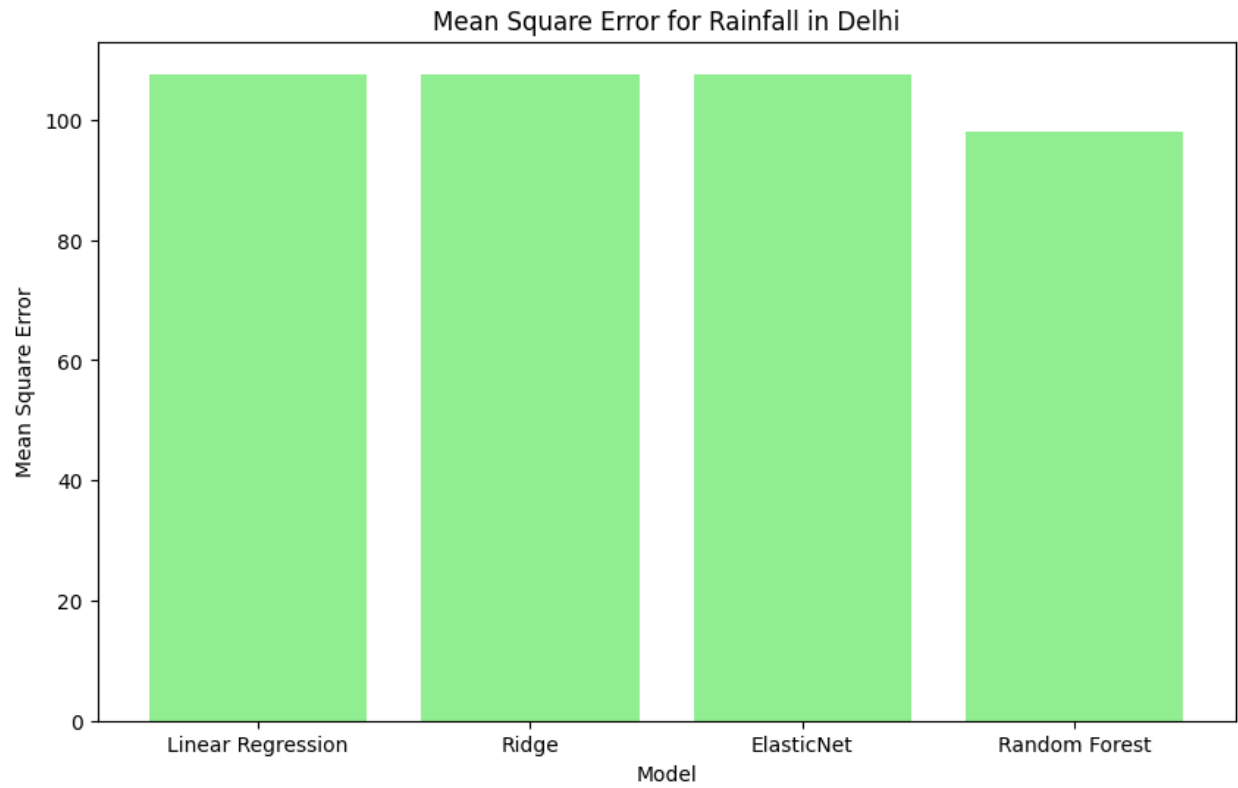
Model Evaluation for Temperature in Delhi



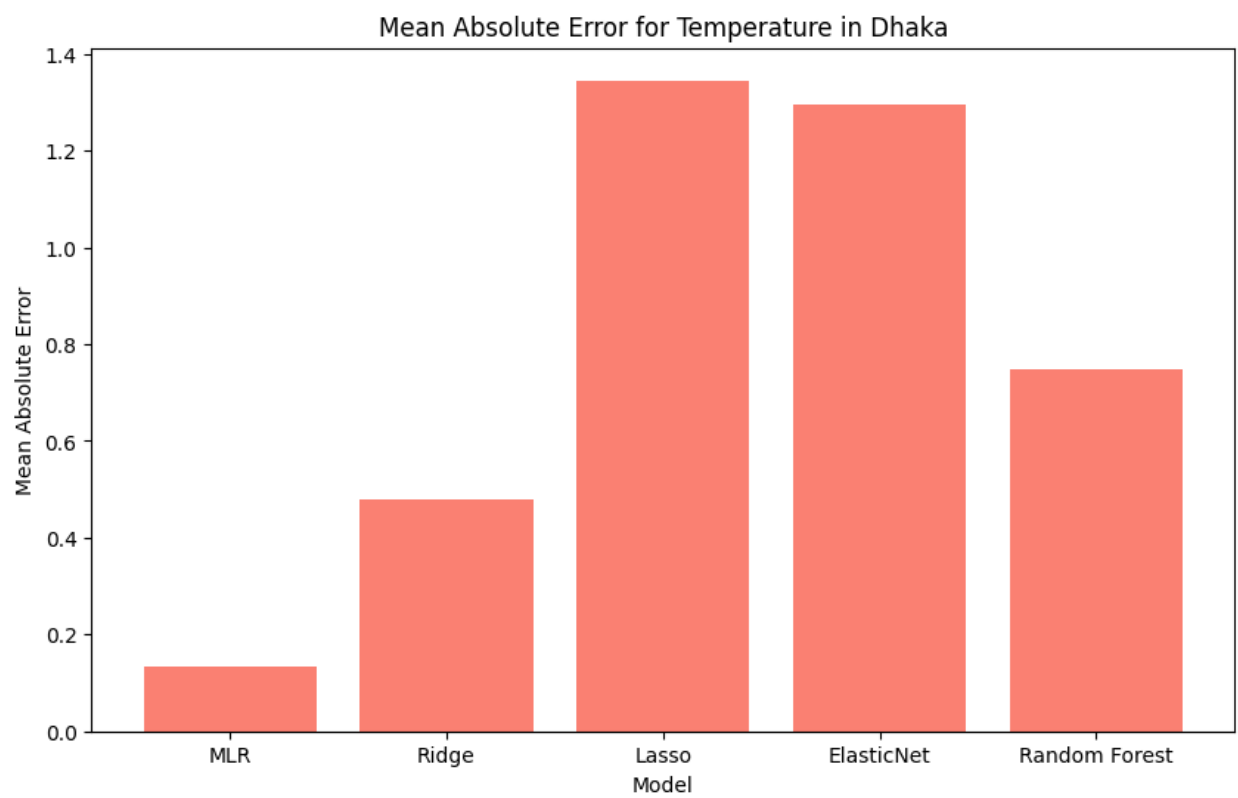
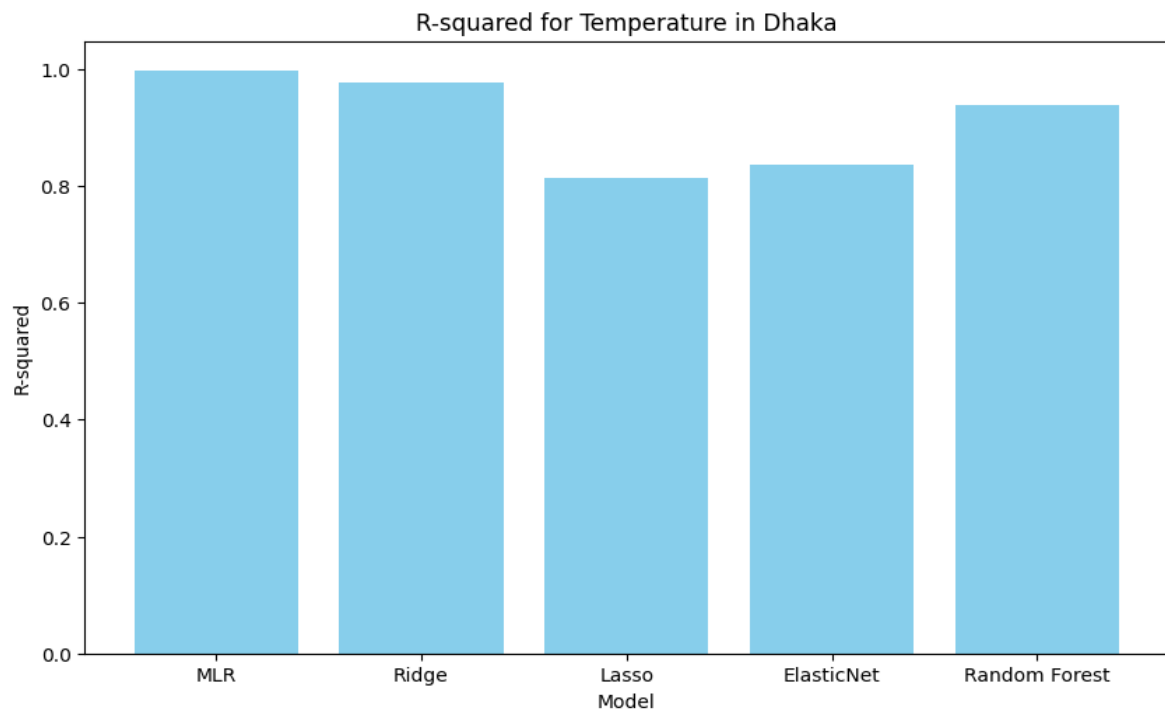


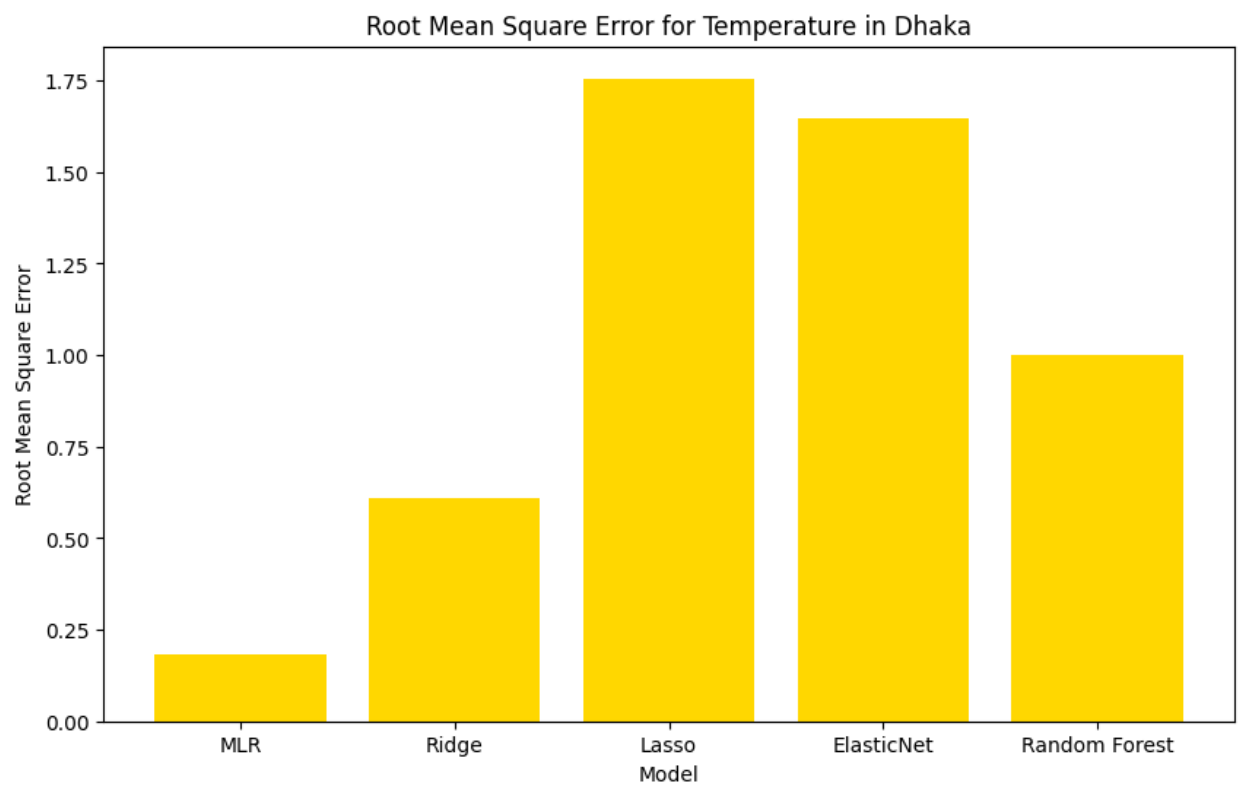
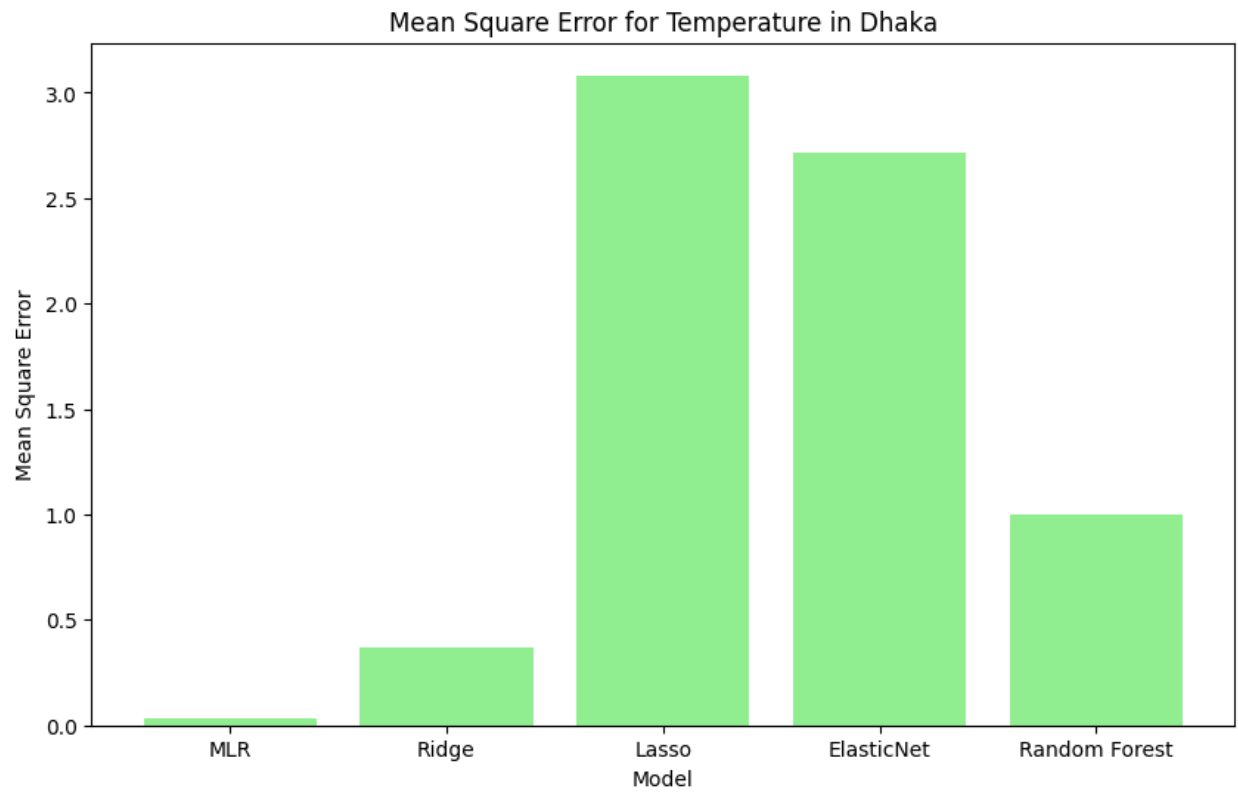
Model Evaluation for Rainfall in Delhi



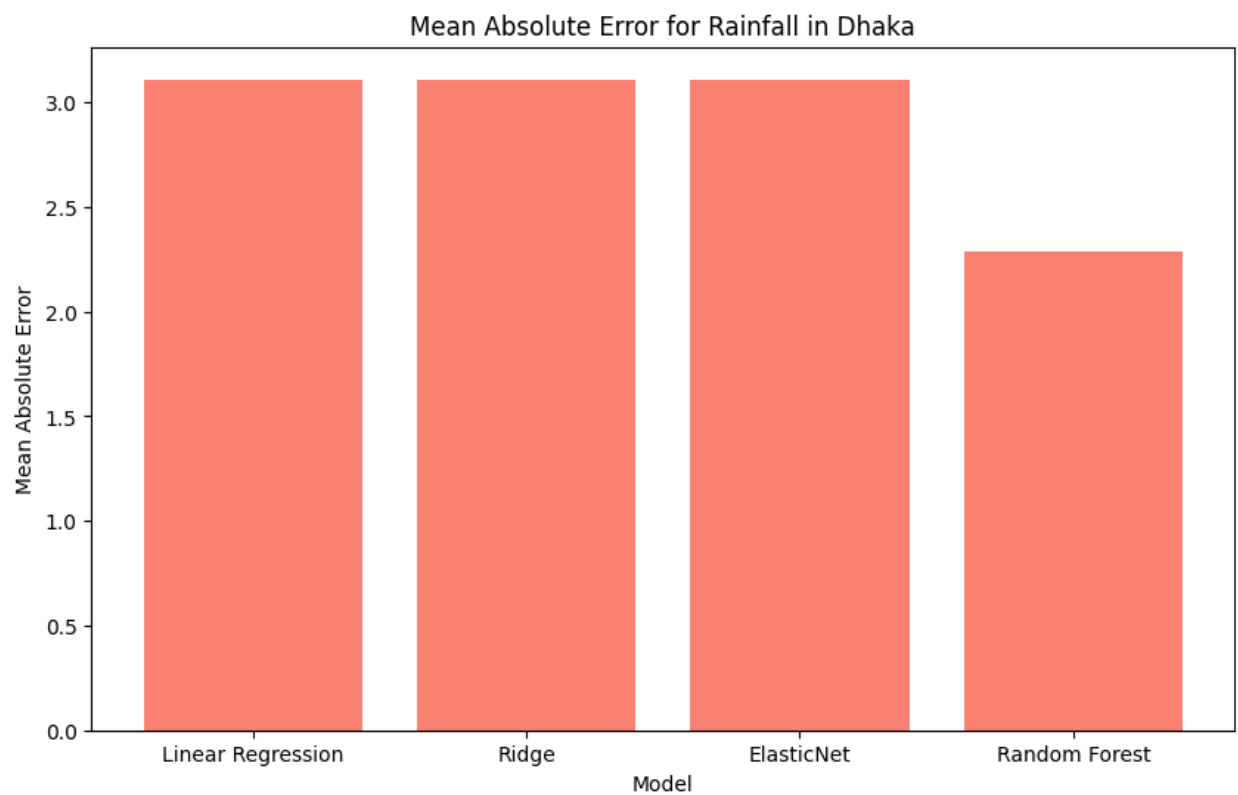
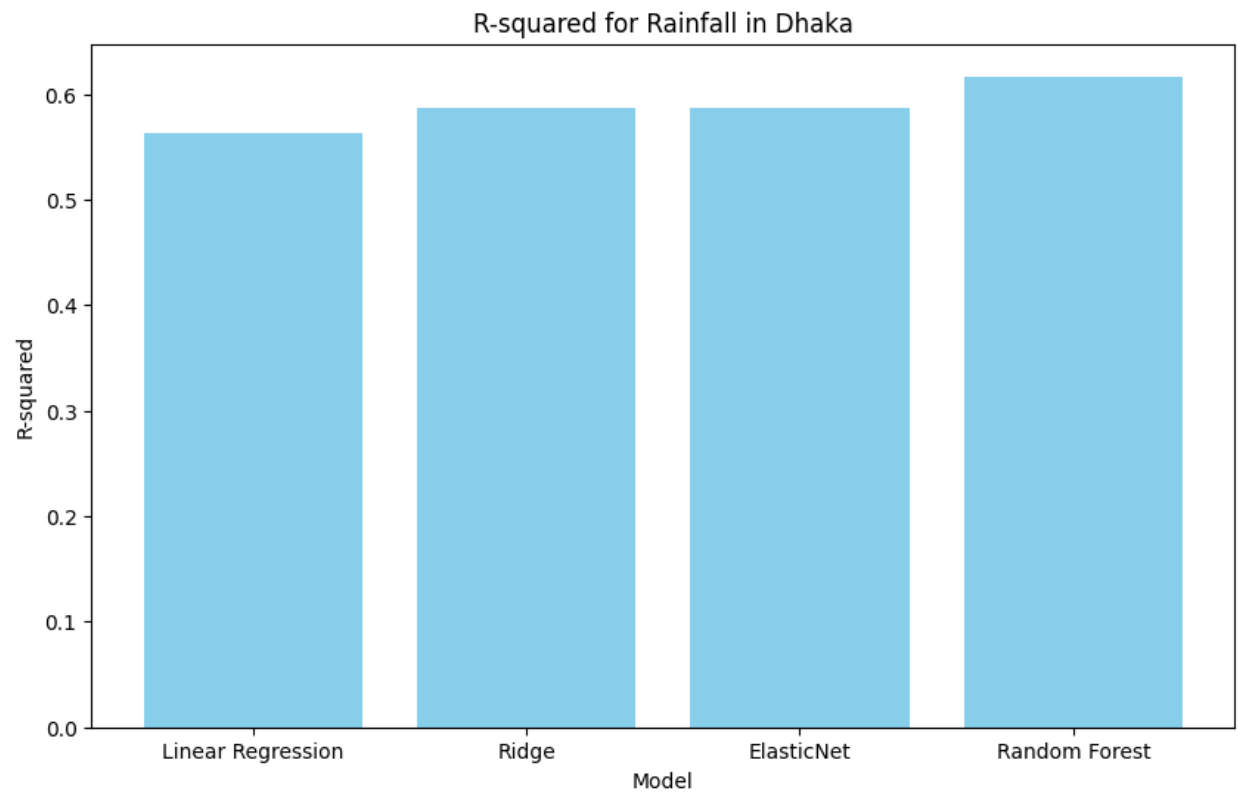


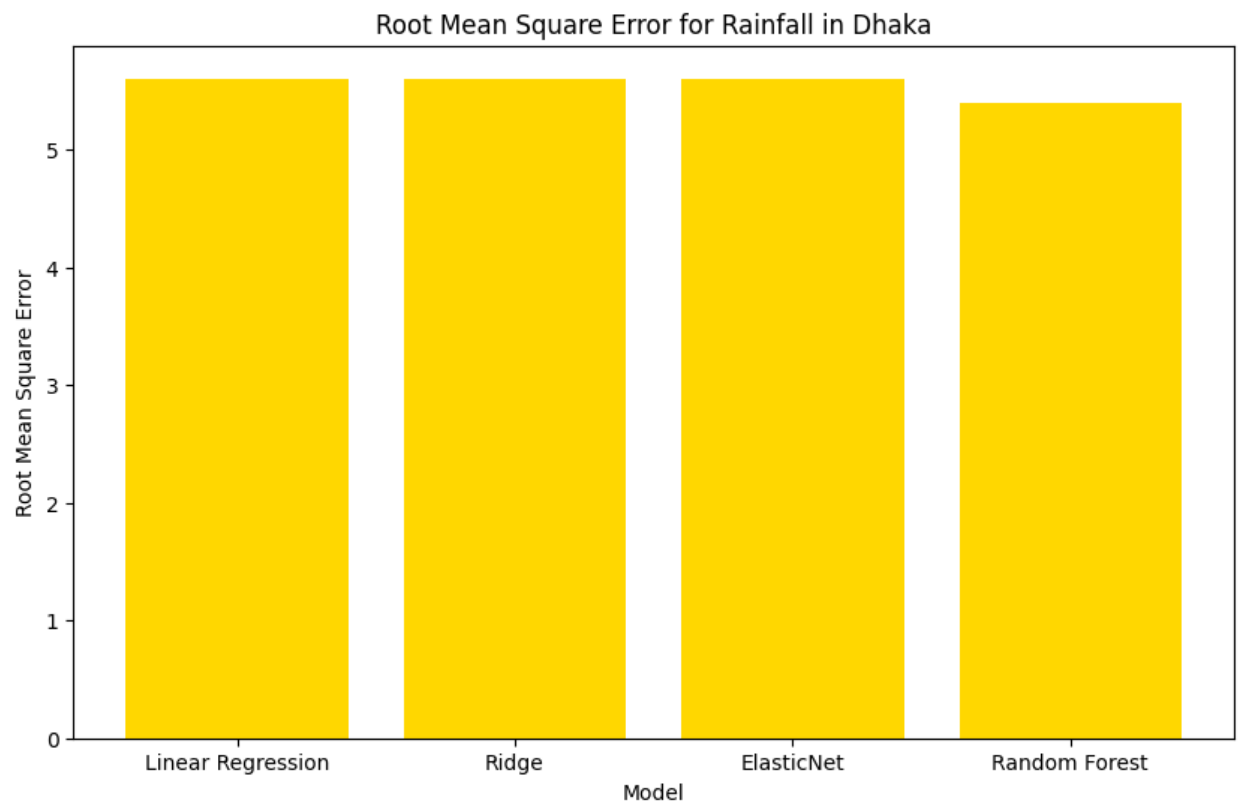
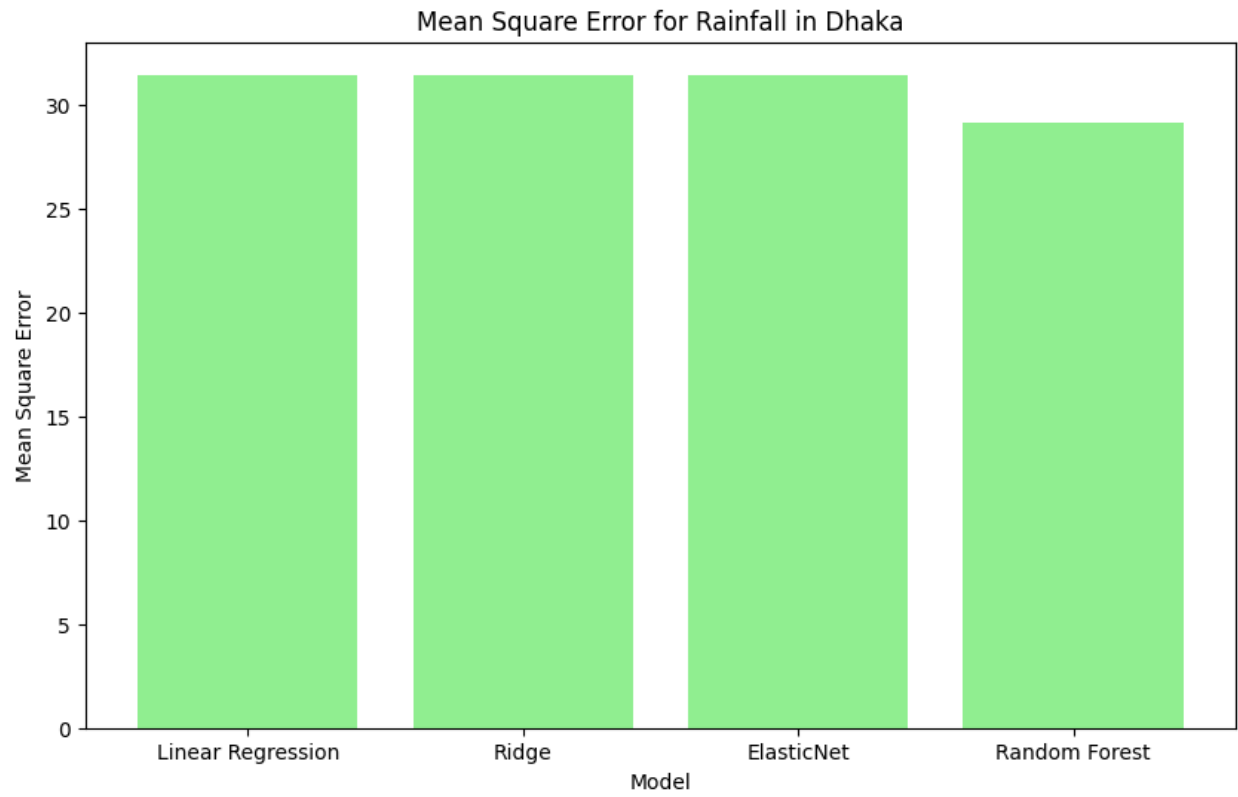
Model Evaluation for Temperature in Dhaka





Model Evaluation for Rainfall in Dhaka

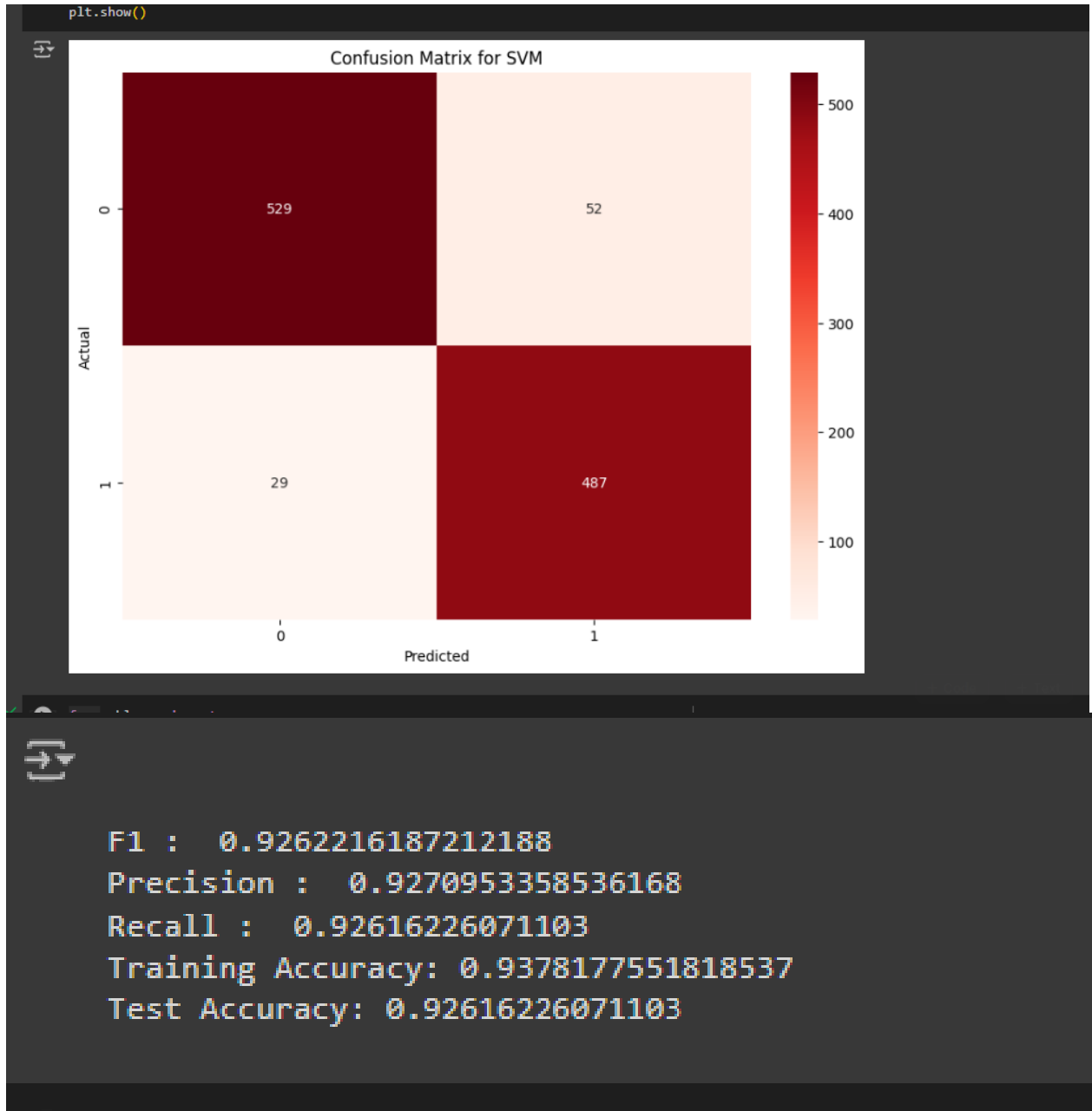




Classification Model Comparisons:

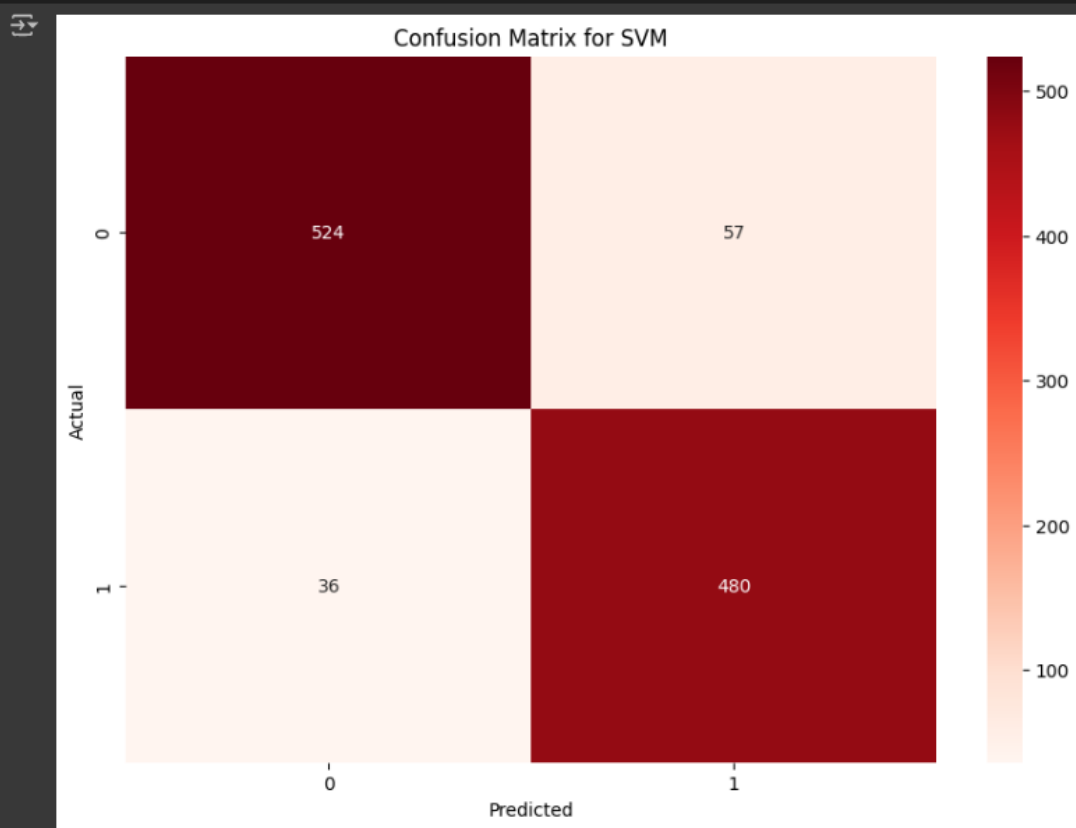
SVM Confusion Matrix With 5, 10, 15 Best Features Selected :

15 best :



Columns : ['tempmax', 'tempmin', 'temp', 'feelslikemin', 'feelslike', 'dew',
'humidity', 'precip', 'precipcover', 'preciptype', 'windgust',
'windspeed', 'sealevelpressure', 'visibility', 'uvindex']

10 Best :



F1 : 0.9152885343723716

Precision : 0.9160246584132448

Recall : 0.9152233363719234

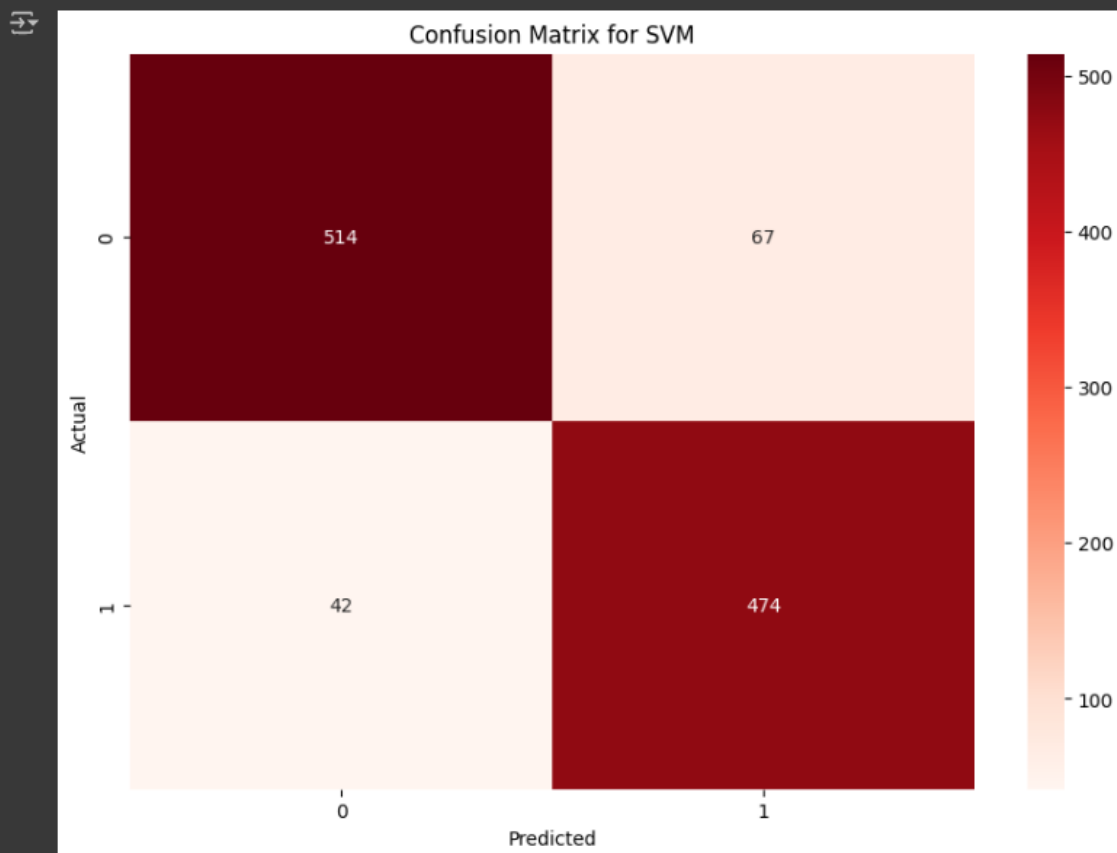
Training Accuracy: 0.9178725068439577

Test Accuracy: 0.9152233363719234

Columns: ['tempmax', 'temp', 'feelslikemin', 'feelslike', 'dew', 'humidity', 'precipcover', 'preciptype', 'visibility', 'uvindex']

5 Best :

```
plt.title('Confusion Matrix for SVM')  
plt.show()
```



F1 : 0.9007207808643541

Precision : 0.9017389508576229

Recall : 0.9006381039197813

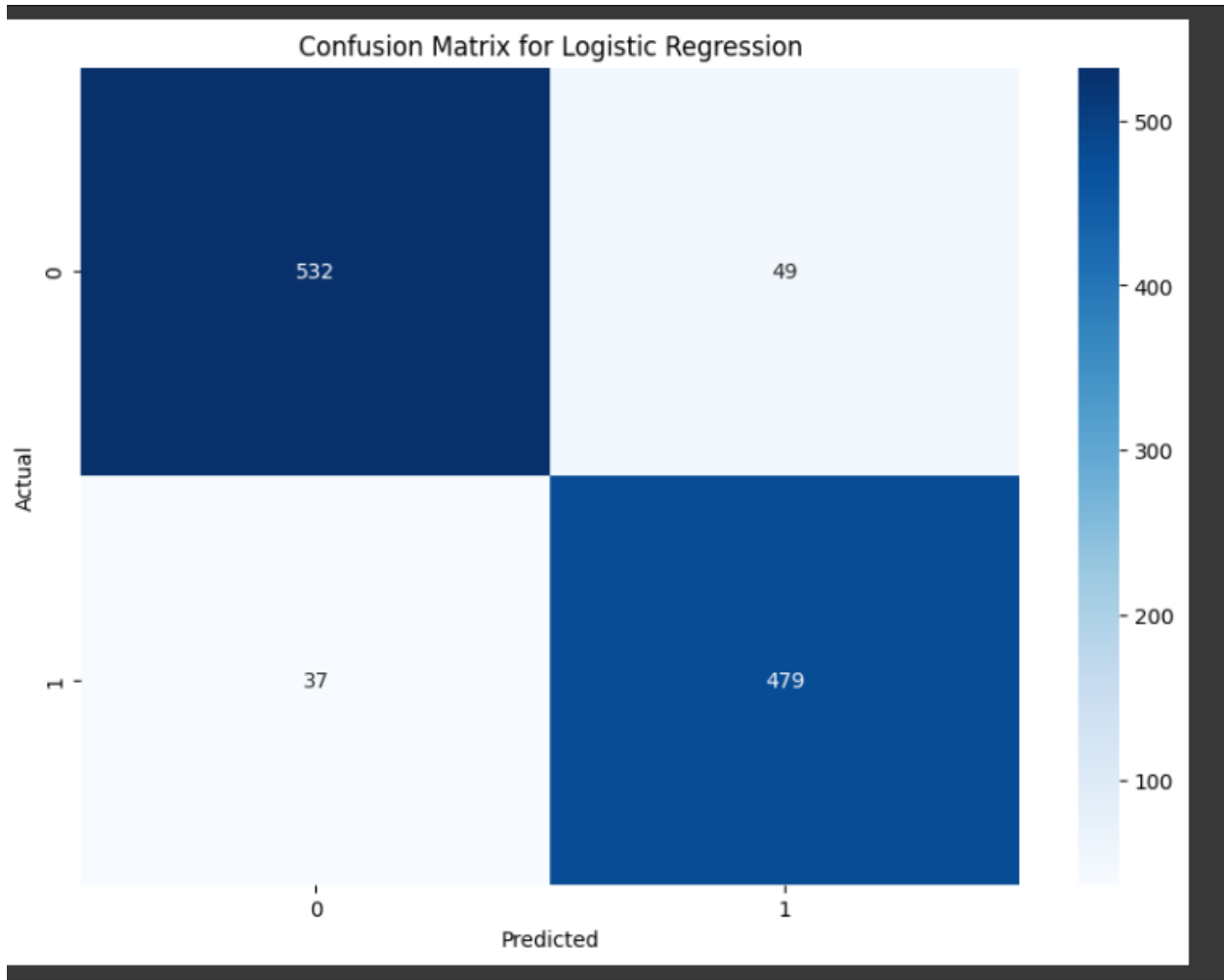
Training Accuracy: 0.8955807587016035

Test Accuracy: 0.9006381039197813

Columns : ['temp', 'feelslike', 'dew', 'humidity', 'visibility']

LogisticRegression Confusion Matrix With 5, 10, 15 Best Features Selected :

15 Best :

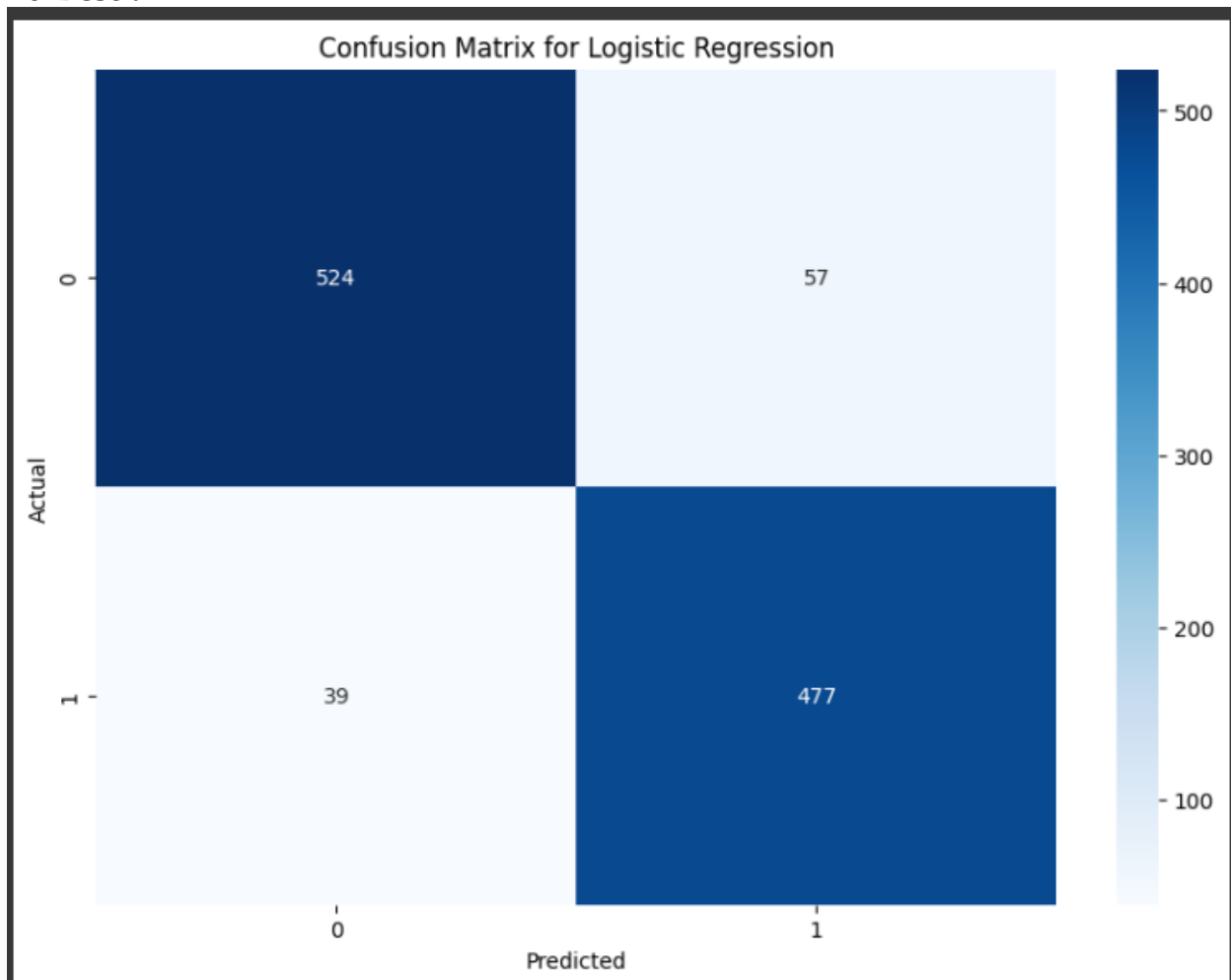


```
F1 : 0.9216459044862255
Precision : 0.9219082224422721
Recall : 0.9216043755697356
Training Accuracy: 0.932733672272194
Test Accuracy: 0.9216043755697356
```

Columns : ['tempmax', 'tempmin', 'temp', 'feelslikemax', 'feelslikemin', 'feelslike', 'dew', 'humidity', 'precip', 'precipprob', 'precipcover', 'preciptype', 'windgust', 'windspeed', 'winddir', 'sealevelpressure',

'cloudcover', 'visibility', 'uvindex', 'conditions']

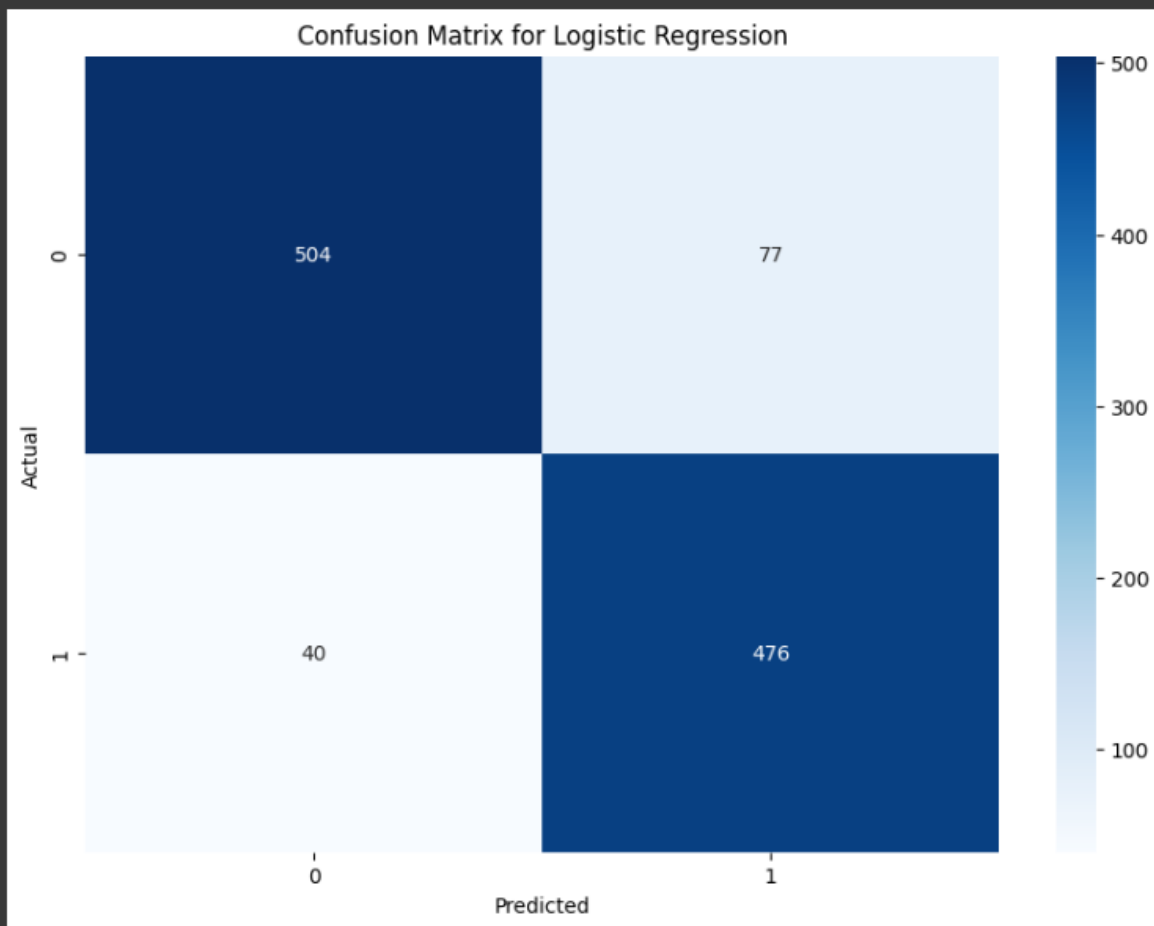
10 Best :



```
F1 : 0.91255023918652
Precision : 0.9131034244874583
Recall : 0.9124886052871468
Training Accuracy: 0.9163081736409855
Test Accuracy: 0.9124886052871468
```

Columns : ['tempmax', 'temp', 'feelslikemin', 'feelslike', 'dew', 'humidity',
'precipcover', 'preciptype', 'visibility', 'uvindex']

5 Best :



F1 : 0.8934373651074919

Precision : 0.8955618144930565

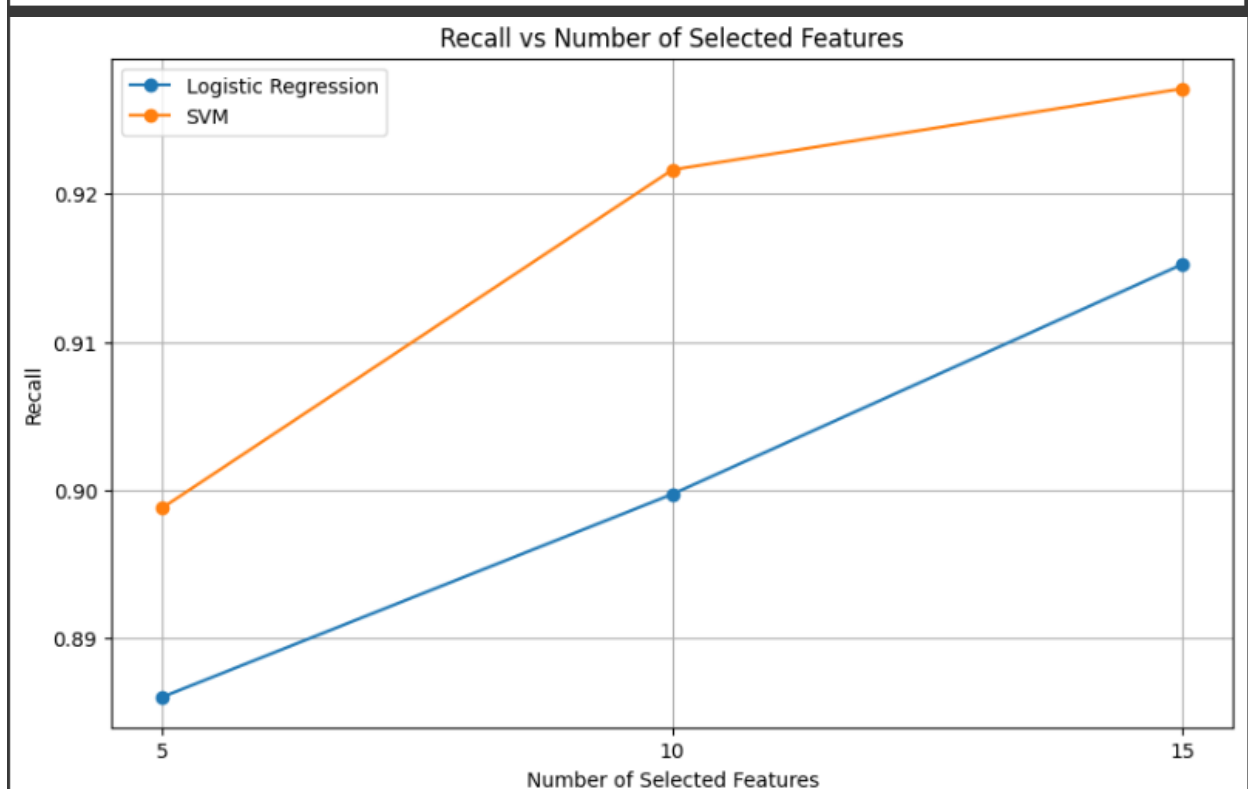
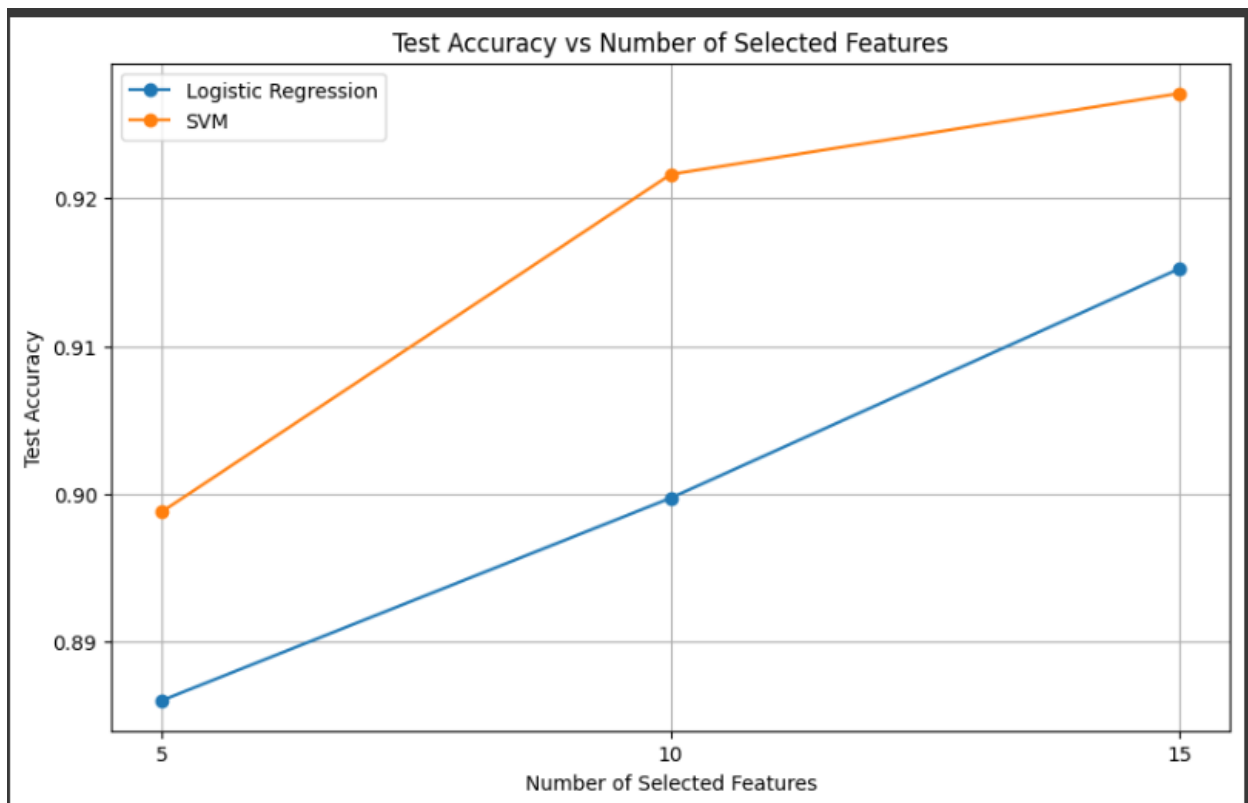
Recall : 0.8933454876937101

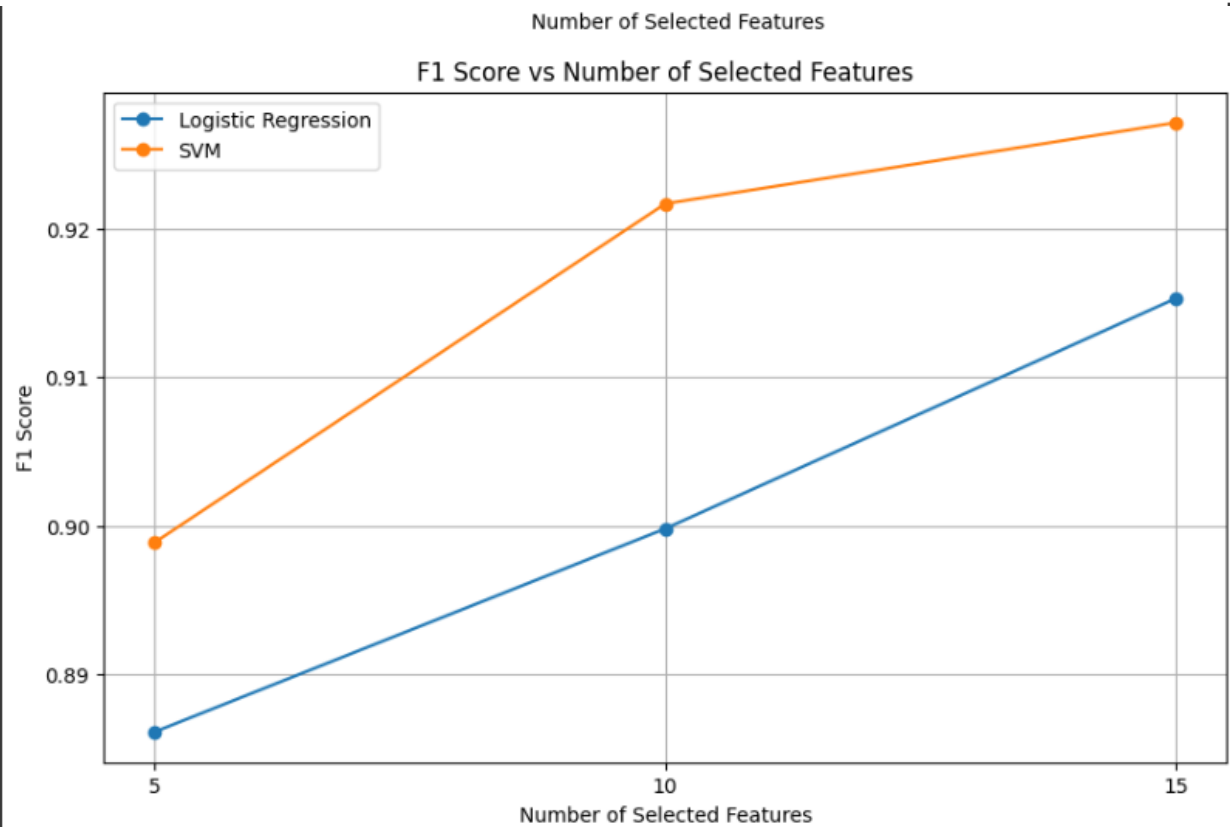
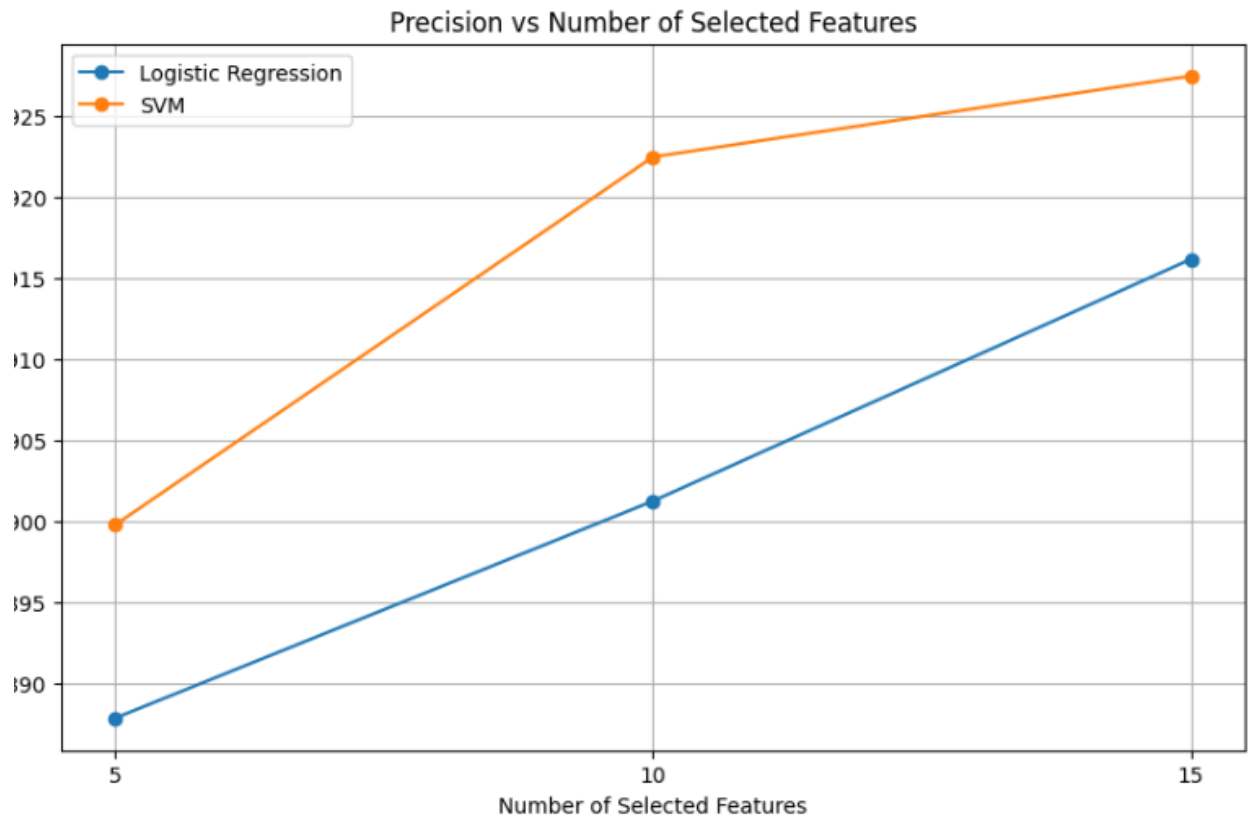
Training Accuracy: 0.8920610089949159

Test Accuracy: 0.8933454876937101

Columns :

['temp', 'feelslike', 'dew', 'humidity', 'visibility']





7. Discussion

After working with different models, I think random forest is better than other linear regression models.

1. **Model Complexity:** Random Forest is a more complex model compared to Linear Regression. It can capture nonlinear relationships between features and target variables more effectively
2. **Sensitiveness to Outliers:** Linear Regression is sensitive to outliers, which significantly affect the model's performance by skewing the regression line. On the other hand, Random Forest is less affected by outliers due to its ensemble nature. It aggregates predictions from multiple decision trees, which reduces the impact of outliers on the overall model.
3. **Handling of Nonlinear Relationships:** Random Forest can handle nonlinear relationships between features and the target variable better than Linear Regression.
4. **Feature Importance:** Random Forest provides a measure of feature importance, which helps in identifying the most influential features for prediction.
5. **Resilience to Overfitting:** Random Forest is inherently resistant to overfitting, especially when using a large number of trees in the ensemble. Each tree in the forest is trained on a random subset of data and features, which promotes diversity and reduces overfitting. In contrast, Linear Regression can be prone to overfitting if the model is dealing with high-dimensional data.

Classification models:

From the charts we can see that both models perform similarly, but the SVM is slightly better. This may be due to :

1. The non linearity in data : SVM can handle nonlinear data in a dataset better than a logistic regression model.
2. SVM can maximize the decision boundary more which might have led to a slightly better model.

3. The parameter tuning in SVM might have been better than SVM.

8. Conclusion

The project was a great challenge for us but we could learn a lot from doing the project.

We have gone through a little hard time trying to build the classification models and implement all those regression models but we overcame most of the challenges and did our best to implement and show everything that was required and present the project beautifully.

There were a lot more we wanted to do with this dataset but the operations ran slow due to the huge size. There would be more scope for experimentation if we had better hardware.